electronics

MDPI

Article

Spec2SeqFuzz: A Category Prediction-Guided Approach for Stateful Multi-Step REST API Fuzzing

Zhuofeng He [1,2], Sunpei Shang [1,2,3,*], Yumeng Guo [2,*] and Aojie Zhou [1,2]

[1] Business School, Central South University, Changsha 410083, China; zhuofeng@csu.edu.cn (Z.H.); 231602017@csu.edu.cn (A.Z.)
[2] Xiangjiang Laboratory, Changsha 410205, China
[3] School of Artificial Intelligence and Advanced Computing, Hunan University of Technology and Business, Changsha 410205, China
* Correspondence: sspralph@csu.edu.cn (S.S.); yumeng.guo@xjlab.org.cn (Y.G.)

Abstract

REST APIs have become a dominant interface for modern web applications and cloud services, and a growing body of work has studied automated testing and reproducible error discovery for such systems. Prior approaches have explored dependency inference, cross-request value reuse, and, more recently, learning- or LLM-based test generation. However, deep stateful multi-step reproducible error discovery remains difficult in practice because sequence construction is still often performed directly in the endpoint space, reusable runtime artifacts are not always tightly coupled with sequence expansion, and online LLM-driven generation may introduce cost and instability. We present Spec2SeqFuzz, a stateful multi-step fuzzing framework for REST API systems. The central idea is to guide online exploration in a compact category space rather than directly in the full endpoint space. Spec2SeqFuzz uses LLMs only in an offline pre-processing stage to normalize public multi-step PoCs, classify OpenAPI endpoints into a transferable category taxonomy, and construct training data for next-category prediction. During online fuzzing, the framework predicts the next likely API category from the executed prefix and observed response feedback, maps the predicted categories back to concrete endpoints, and combines this guidance with black-box endpoint fuzzing, proxy-based payload collection, and snapshot-assisted state restoration. We implemented a prototype and evaluated it on GitLab and WordPress, using MINER as the primary reproduced baseline in our current study. The results show that Spec2SeqFuzz is promising for both multi-step and single-endpoint error discovery on these two targets. Following the terminology used in MINER, we report reproducible errors rather than treating every triggered failure as a confirmed security vulnerability. Across the two targets, Spec2SeqFuzz discovers more reproducible multi-step errors than MINER, while the ablation results further suggest that category guidance, payload reuse, and depth-first stateful exploration are important to the final error-discovery performance.

Keywords: REST API fuzzing; stateful multi-step fuzzing; category prediction; OpenAPI; large language models; error discovery

Check for updates

Academic Editor: Arkaitz Zubiaga

Received: 22 April 2026
Revised: 22 May 2026
Accepted: 22 May 2026
Published: 26 May 2026

Copyright: © 2026 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license.

1. Introduction

Modern web applications and cloud services increasingly expose their core functionality through REST APIs described by OpenAPI specifications, turning resource management, access control, configuration updates, file handling, and background-task management into programmatically accessible HTTP endpoints [1,2]. This trend improves interoperability

and automation, but it also enlarges the attack surface of real systems. In many cases, faults or abnormal behaviors do not arise from a single isolated request. Instead, they emerge only after a sequence of dependent operations drives the target into a specific state, after which abnormal behavior becomes observable [3–5]. Typical examples include create-then-use patterns, privilege or configuration-dependent workflows, and multi-step resource-lifecycle inconsistencies.

This observation has motivated a substantial body of research on automated REST API testing and fuzzing. Existing work has shown the importance of specification parsing and automated request generation for REST API testing [6,7]. Other studies further investigate dependency recovery, producer–consumer relationships, and request-sequence construction for state-aware REST API fuzzing [8–10]. More recent approaches also explore advanced sequence generation, value propagation, and state-aware execution strategies [11,12]. Some systems further introduce learning or language-model assistance into REST API testing [13,14].

Our work therefore does not claim to be the first to reuse cross-request values, to consider operation dependencies, or to apply learning to REST API testing. Instead, our contribution lies in how these ingredients are combined for the setting we target in this paper: stateful multi-step fuzzing for discovering reproducible errors and failure-triggering behaviors. We argue that, for this setting, directly organizing search in the endpoint space is often too expensive and unstable, especially when the target exposes a large number of operations.

Spec2SeqFuzz addresses this issue by introducing a category-level abstraction for online exploration. Rather than predicting the next concrete endpoint, the framework first predicts the next likely API category from the executed prefix and response feedback, and only then maps that category back to candidate endpoints. This abstraction compresses the search space and provides more transferable guidance across targets than endpoint-level enumeration alone. In our design, LLMs are used only offline, where they normalize public multi-step PoCs, classify endpoints into a unified taxonomy, and help construct training samples. The online path remains lightweight: a learned next-category predictor guides exploration, black-box fuzzers generate concrete endpoint-level inputs, and a proxy layer collects reusable runtime fields for later requests.

A second design point is that payload reuse is treated as part of the exploration loop rather than as an isolated post-processing feature. In many multi-step REST API workflows, later requests depend on dynamic fields such as IDs, handles, tokens, slugs, or configuration keys returned by earlier successful responses [15]. Without carrying such runtime artifacts across steps, even a semantically reasonable sequence often fails before it reaches an anomalous state. Spec2SeqFuzz therefore couples category-guided expansion with proxy-based payload collection and spec-constrained parameter filling.

A third design point is the integration of stateful scheduling with environment rollback. Deep exploration in real systems is easily affected by state contamination across branches. To reduce this effect, Spec2SeqFuzz combines depth-first exploration with virtual-machine snapshot restoration, allowing sibling branches to be explored from a reproducible state.

We implemented a prototype and evaluated it on two large open-source REST API systems, GitLab and WordPress, using MINER as the primary reproduced baseline in the current version of the paper. The current evaluation is intentionally conservative in its claim scope: our goal is to show that category-guided stateful fuzzing is effective and promising on complex targets, not to claim a definitive state-of-the-art result over all recent REST API testing systems. A broader comparison against additional recent tools remains future work.

We clarify the novelty of Spec2SeqFuzz as follows. We do not claim that dependency inference, payload reuse, DFS exploration, black-box fuzzing, or LLM-assisted pre-processing is new in isolation. The key novelty lies in formulating stateful REST API fuzz-sequence construction as a next-category prediction problem. Existing REST API fuzzers typically extend request sequences in the concrete endpoint space, dependency graph, or historical sequence-template space. In contrast, Spec2SeqFuzz projects endpoints into a shared semantic category space, learns category-transition patterns from public multi-step PoCs, and predicts the next likely API category based on the executed prefix and observed response feedback. This prediction is then used to guide online sequence expansion before mapping categories back to concrete endpoints. Therefore, Spec2SeqFuzz introduces a category prediction-guided sequence-construction paradigm, rather than merely combining existing fuzzing components.

Our main contributions are summarized as follows.

We propose Spec2SeqFuzz, a category-guided stateful multi-step fuzzing framework for REST API systems. The framework uses category-level guidance to steer online exploration and alleviate the combinatorial explosion of endpoint-level search.

We design an LLM-assisted offline knowledge-construction pipeline. By leveraging public PoCs and OpenAPI specifications, our method constructs training samples, endpoint-category mappings, and high-value seed endpoints, providing transferable prior knowledge for sequence guidance.

We develop a closed loop that integrates black-box fuzzing with runtime payload reuse. By combining the endpoint-level strength of public black-box fuzzers with proxy-based dynamic field collection, Spec2SeqFuzz enables later requests to reuse critical parameters extracted from prior responses and thereby improves executability along deep stateful paths.

We provide an initial empirical study on real-world open-source systems. Our results show that Spec2SeqFuzz is promising on two complex targets in terms of reproducible error discovery, and our ablation study further demonstrates the importance of category guidance, payload reuse, and stateful scheduling.

In the spirit of open science, we make our tool available at https://github.com/horis-dev/Spec2SeqFuzz (accessed on 6 March 2026).

2. Background

2.1. RESTful APIs and OpenAPI-Based Testing

REST, as an architectural style for networked applications, has become one of the dominant paradigms through which modern Web services and cloud platforms expose their functionality [16]. In RESTful systems, resources are typically accessed and manipulated through standard HTTP methods, where GET, POST, PUT, and DELETE correspond to common operations such as retrieval, creation, update, and deletion. As Web systems have become increasingly interface-driven, more and more core business logic—including authentication, resource management, configuration updates, file handling, and asynchronous job scheduling—has been encapsulated into programmatically accessible API endpoints. As a result, RESTful APIs have evolved into a primary interface through which system functionality is delivered and, consequently, a critical surface for security exposure.

To support automated testing, prior work has widely relied on OpenAPI/Swagger specifications to describe endpoint paths, request parameters, response formats, and object schemas. OpenAPI provides a unified and structured interface description for RESTful APIs, allowing testing tools to parse request templates, parameter types, and dependency information from the specification and then generate executable HTTP requests accordingly [17]. Existing studies have shown that OpenAPI-based automated testing has become a major

direction in REST API testing. Some representative techniques build on OpenAPI specifications to support standardized request generation and black-box API testing [18,19]. Other systems use OpenAPI specifications to infer dependencies and construct state-dependent request sequences [10,12]. More recent studies further combine OpenAPI-based testing with learning-based or language-model-assisted test generation [13,14].

2.2. Stateful Request Sequences and Multi-Step Vulnerabilities

Although OpenAPI specifications can provide endpoint-level syntactic information, many API faults are not triggered by a single isolated request [20]. Instead, many faults and security vulnerabilities depend on a sequence of API operations with explicit preconditions and postconditions: earlier requests create resources, establish sessions, write configurations, or return object identifiers, while later requests continue advancing execution based on these runtime states and eventually expose anomalous behavior under a specific context. RESTler abstracts such relationships as producer–consumer dependencies and leverages dependency inference to generate stateful request sequences for exploring deeper service states [10]. MINER further shows that the effectiveness of REST API fuzzing hinges on two closely related factors: whether the fuzzer can construct sufficiently long valid sequences, and whether it can generate high-quality requests that pass server-side validation. Otherwise, fuzzing is likely to remain trapped in shallow states, making it difficult to reach complex logic flaws and deeper state-dependent failures [21].

NAUTILUS reinforces this observation from an empirical vulnerability perspective. Its study reports that, among 609 analyzed RESTful API vulnerabilities, 499 (approximately 82%) require multiple API operations to be triggered [22]. This suggests that multi-step vulnerabilities are not a marginal phenomenon, but rather a common pattern in RESTful services. Similarly, VOAPI2 observes that scanners focusing only on single endpoints are often unable to construct multi-request sequences that satisfy data dependencies and protocol-state constraints, and therefore struggle to uncover vulnerabilities such as XSS, SSRF, and path traversal that are hidden behind complex interactions [23–25]. In essence, security testing for RESTful APIs is not merely an input-mutation problem; it is equally a problem of state progression and sequence organization [26].

2.3. LLMs in Fuzzing

In recent years, large language models (LLMs) have demonstrated strong capabilities in code understanding, natural-language analysis, and program generation, and have increasingly been introduced into software testing and fuzzing tasks [27,28]. In REST API testing, this trend has led to several new directions, including learning-based test generation, online model-guided request construction, and agent-style exploration, as exemplified by DeepRest, LlamaRestTest, and AutoRestTest [29–31]. These efforts suggest that language models can help bridge the gap between syntactic API descriptions and semantically meaningful test behaviors.

At the same time, existing learning- or LLM-based REST API testing systems differ substantially in how they place model intelligence in the workflow. Some approaches use online model inference to directly generate or rank concrete requests and operation sequences. Others use LLMs as part of broader exploration frameworks. These designs can be powerful, but they may also introduce higher runtime cost, more online decision complexity, and greater sensitivity to prompt quality or model instability.

Spec2SeqFuzz follows a different design choice. We use LLMs only in the offline stage to preprocess public PoCs, classify OpenAPI endpoints into semantic categories, and construct category sequence training samples. The online stage does not rely on an LLM to generate each next request. Instead, it uses a lightweight next-category prediction

model for sequence guidance, black-box fuzzers for endpoint-level input generation, and a proxy layer for runtime value collection and reuse. In other words, our goal is not to compete with online LLM generation directly, but to exploit LLMs as an offline semantic abstraction mechanism that compresses the endpoint-level search space while keeping the online execution path efficient and reproducible.

This distinction is important for understanding our contribution. We do not claim that Spec2SeqFuzz is the first REST API testing system to incorporate learning or language-model assistance. Rather, our contribution aims to show that offline semantic abstraction, category-guided online exploration, and runtime payload reuse can be combined into a practical workflow for stateful multi-step reproducible error discovery.

3. Design of Spec2SeqFuzz

3.1. Framework of Spec2SeqFuzz

The overall architecture of Spec2SeqFuzz consists of two complementary modules: offline knowledge construction and online stateful exploration (as illustrated in Figure 1). The offline module is responsible for building transferable prior knowledge and predictive models, while the online module performs stateful fuzzing under this guidance and further improves search efficiency through a feedback loop. The two modules are connected through three key assets: category mappings, prediction models, and the payload pool.

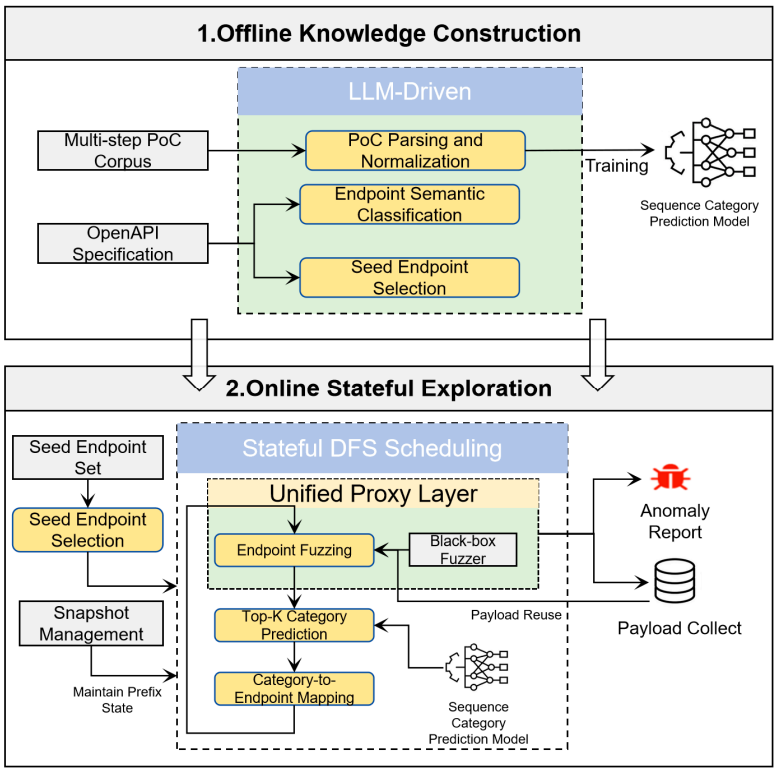


Figure 1. The overall architecture of Spec2SeqFuzz.

The goal of the offline stage is to construct the knowledge and models required for multi-step sequence generation from public PoCs and OpenAPI specifications with the assistance of LLMs. It includes the following components:

- PoC serialization and annotation. Multi-step PoCs are normalized into structured templates and further transformed into training samples.
- OpenAPI endpoint-category knowledge. Each endpoint in the OpenAPI specification is classified into a semantic category, producing an endpoint-to-category mapping.

• Seed-endpoint set. Initial endpoints with fewer dependencies and a higher likelihood of producing critical runtime parameters are selected as seed endpoints.

The online stage takes these offline assets as priors and performs multi-step fuzzing through stateful DFS-based exploration. Its core workflow includes the following steps:

• Category prediction (Top-k). Given the category and status-code sequence of the executed prefix, the model predicts the Top-k candidate categories for the next step together with their probabilities, thereby narrowing the branching space.

• Stateful DFS scheduling. Treating each executed sequence prefix as a state, the scheduler retrieves candidate endpoints by reverse-indexing the predicted Top-k categories, filters them according to the payload pool and required REST API parameters, invokes black-box fuzzers for execution, and then performs depth-first exploration toward anomalous behaviors.

• Snapshot rollback. Before exploring each branch, the system restores a baseline snapshot and, when necessary, creates or deletes intermediate snapshots. This design mitigates state pollution and ensures reproducibility of the same sequence across runs.

• Black-box tool collaboration and proxy-based collection. Public black-box fuzzers are responsible for endpoint-level input generation and anomaly triggering, while the proxy layer handles traffic collection, payload extraction and reuse, and anomaly report generation.

3.2. Offline Knowledge Construction

3.2.1. API Category System

To support sequence modeling and online guidance for stateful multi-step fuzzing, we introduce a semantics-driven API category system. The goal of this taxonomy is not to provide a universal ontology for all REST APIs, but to construct a compact and transferable abstraction that is useful for sequence prediction and exploration. In particular, we want the label space to be expressive enough to capture common state-transition roles across services, while remaining small enough to support stable learning and low-cost online prediction.

Our taxonomy is derived from two sources of regularity. The first is the recurring state-transition structure observed in modern web/API systems, such as authentication and session establishment, resource creation and subsequent read/update/delete operations, search and listing behavior, file or data movement, configuration changes, and asynchronous task execution. The second is the recurring multi-step attack logic observed in public PoCs, where earlier requests often prepare objects, identities, permissions, or runtime artifacts that are consumed by later requests. We therefore group endpoints by their functional role in state progression and dependency propagation, rather than by surface-level URL similarity alone.

This design is guided by three practical considerations. First, the taxonomy should preserve the distinctions that matter most for multi-step reachability, such as whether an operation creates a reusable object, reads key fields needed later, mutates existing state, or triggers a background action. Second, it should generalize across systems with different endpoint names and resource structures. Third, it should remain simple enough for one-label-per-endpoint annotation, which reduces ambiguity in both manual review and model training.

Table 1 summarizes the category set used in this paper. Each endpoint is mapped to one primary category. We intentionally adopt a single primary label, even though some endpoints may exhibit mixed or multi-functional semantics, because the online next-category predictor requires a compact and stable output space. This is therefore a pragmatic modeling choice rather than a claim that endpoint behavior is always strictly mono-functional.

Table 1. The category set used in Spec2SeqFuzz. Each endpoint is mapped to one primary category.

High-Level Category	Fine-Grained Category	Description
Authentication and Session	Auth	Endpoints for login, logout, session creation, or session termination.
	UserRegister	Account lifecycle endpoints such as registration and password reset.
Resource Operations	ResourceCreate	Endpoints that create new referenceable entities.
	ResourceRead	Endpoints used to retrieve key fields and confirm resource state.
	ResourceUpdate	Endpoints that update existing resources.
	ResourceDelete	Endpoints that delete existing resources.
Search and Listing	Search	Advanced query endpoints with filters, keywords, sorting, and related options.
File and Data Flow	FileUpload	File upload endpoints.
	FileDownload	Endpoints for file retrieval or export.
Configuration and System	ConfigWrite	Write operations on system configuration and related settings.
	BackgroundJob	Endpoints that trigger asynchronous tasks or jobs.
	HealthCheck	Healthcheck, metrics, or probe endpoints.
External Service Interaction	APIGateway	Endpoints that proxy or forward requests to external APIs.
Special/Error-prone	HiddenAPI	Undocumented or non-exposed endpoints.

For ambiguous endpoints, we assign the primary category according to the endpoint's dominant role in stateful sequence construction. The decision is based on the HTTP method, path semantics, operation description, parameters, request-body schema, response schema, and the endpoint's likely role in dependency propagation.

We also emphasize that the taxonomy itself is not the final contribution in isolation. Similar functional regularities have implicitly or explicitly appeared in prior API testing work. Our contribution is to operationalize such regularities as a reusable category space for next-step prediction and category-guided exploration. A broader quantitative study of alternative taxonomies is beyond the scope of the current paper and remains an important direction for future work.

3.2.2. LLM-Based Extraction of Multi-Step PoC Sequences

To build training data for sequence category prediction, we systematically collect public multi-step attack PoCs and transform them into structured, labeled sequences. Our pipeline consists of four stages: (1) PoC collection and filtering; (2) sequence normalization and extraction; (3) LLM-assisted sequence annotation; (4) manual cross-validation.

PoC Collection and Filtering. We collect public PoCs from widely used security-scanner repositories, including xray, nuclei, goby, and well-maintained popular PoC repositories on GitHub [32–34]. These sources provide relatively uniform template formats, explicit rule structures, and frequent updates, making them suitable for large-scale parsing and automated extraction. In total, we gather approximately 12,000 raw PoC templates.

Because the goal of Spec2SeqFuzz is to learn category-transition patterns for stateful multi-step sequence construction, not all collected PoCs are suitable for training. We therefore apply the following filtering criteria. First, we remove single-step PoCs, because they do not contain cross-request state transitions and cannot provide prefix-to-next-category supervision for the sequence prediction model. Second, we remove duplicated or near-duplicated PoCs. In practice, many public PoC repositories contain repeated templates for the same vulnerability, the same request chain, or slightly modified versions of the same exploit logic. We identify duplicates based on vulnerability name or identifier when available, as well as target service, normalized HTTP method/path sequence, and normalized request structure. Third, we discard templates that cannot be reliably parsed into ordered HTTP request steps, such as entries with incomplete request fields, missing method or path information, unsupported script-only logic, or implicit interactions that cannot be

converted into a structured request sequence. Fourth, we exclude templates whose request ordering is unclear, because the sequence prediction model requires an explicit step order.

After applying these filters, the remaining samples are normalized into structured multi-step request sequences and then manually cross-validated. This process yields 6700 normalized multi-step PoC samples for training the sequence category prediction model. The majority of discarded samples are single-step PoCs or duplicated attack templates, while a smaller portion is removed because the request chain cannot be parsed or ordered reliably. This filtering process ensures that the final training set focuses on reproducible multi-step interaction patterns rather than isolated single-request exploits or repeated variants of the same attack.

Normalization of Multi-step Attack Sequences. For each retained PoC, we convert the template into a unified structured representation containing PoC metadata and a multi-step request sequence, which serves as the basis for subsequent labeling and training-data generation. A representative example is shown below:

```
{
  "vuln_id": "N/A",
  "vuln_name": "Poc Yaml Weiphp Path Traversal",
  "service": "weiphp",
  "steps": [
    {
      "order": 1,
      "method": "POST",
      "path": "/public/material/_download_imgage",
      "params": {
        "query": {
          "media_id": "1",
          "picUrl": "./../config/database.php"
        },
        "body": "\"1\":1\n"
        }
    },
    {
      "order": 2,
      "method": "GET",
      "path": "/public/home/file/user_pics",
      "params": {}
    },
    {
      "order": 3,
      "method": "GET",
      "path": "/public/uploads/picture/{{img}}",
      "params": {}
    }
  ]
}
```

Here, steps denote the multi-request chain. Each step records the HTTP method, path, and parameter information, while preserving the original interaction semantics of the PoC for downstream sequence classification.

LLM-assisted Sequence Annotation. To train the sequence category prediction model, we further transform each PoC's multi-step request chain into a supervised sample consisting of a category sequence (cat_seq) and a status sequence (status_seq). Specifically, we use an LLM to assign each request step to an API category according to the taxonomy defined in Section 3.2.1, yielding training instances in the following form:

```
{
  "vuln_id": "N/A",
  "vuln_name": "poc-yaml-weiphp-path-traversal",
  "service": "weiphp",
  "cat_seq": ["FileUpload", "ResourceRead", "ResourceRead"],
  "status_seq": ["200", "200", "200"],
  "target": "web"
}
```

During annotation, we design a unified prompt that feeds the normalized structured template to the model and requires it to output standardized cat_seq and status_seq. We use either ChatGPT-5.2 or DeepSeek V3.2 for this task [35,36]. The prompt emphasizes two requirements: (1) cat_seq must align with the behavioral type of each API operation; (2) status_seq should reflect the typical response class of each step, which later serves as guidance for the online model, especially in 5xx-triggering scenarios.

Cross-validation and Final Dataset. To ensure annotation quality, all LLM outputs are cross-validated by two Ph.D. students and three M.S. students. The review focuses on whether (1) the assigned category matches the request semantics, (2) the step order preserves the original attack logic of the PoC, and (3) the status-code class is reasonable. After manual verification, we obtain a final dataset of 6700 normalized multi-step PoC samples, which serve as the training foundation for our sequence category prediction model.

3.2.3. REST API Endpoint Generation and Filtering

This section describes how endpoints are generated and filtered from OpenAPI documents to construct endpoint-category knowledge and a pool of seed endpoints for subsequent multi-step fuzzing. These artifacts serve as structured priors and executable entry points for later sequence construction. The overall process consists of two stages: endpoint semantic categorization and seed-endpoint selection.

Endpoint semantic categorization. We feed each endpoint in the OpenAPI document to a large language model and ask it to classify the endpoint according to the API category taxonomy defined in Section 3.2.1. The model outputs a dictionary in the form of "endpoint $\rightarrow$ category," which captures the semantic mapping between concrete endpoints and abstract API categories. This mapping serves as critical prior knowledge in subsequent multi-step sequence prediction and DFS-based state expansion. An example output is shown below:

```
{
  "POST /api/v4/groups/{id}/wikis/attachments": "FileUpload",
  "DELETE /api/v4/groups/{id}/wikis/{slug}": "ResourceDelete",
  "PUT /api/v4/groups/{id}/wikis/{slug}": "ResourceUpdate",
  "GET /api/v4/groups/{id}/wikis/{slug}": "ResourceRead",
  "POST /api/v4/groups/{id}/wikis": "ResourceCreate",
  "GET /api/v4/groups/{id}/wikis": "Search",
  ...
}
```

Seed-endpoint selection. To construct executable fuzzing sequences, the system must first identify a set of stable endpoints that can be invoked independently as sequence starting points. We therefore submit each REST API endpoint to the LLM and ask it to determine, based on the endpoint's parameters, the HTTP method, and textual description, whether it satisfies the following requirements for an initial API:

- It does not depend on dynamic objects created by prior requests and can be executed using only authentication or basic parameters.
- It can produce artifacts required by later endpoints, such as IDs, tokens, or file handles.

Based on these criteria, the LLM returns a subset of endpoint descriptions in the same format as the input. This subset serves two purposes during later multi-step sequence generation: it constrains the choice of starting points and improves state reachability. In practice, these two effects significantly increase both sequence executability and overall scalability. An example is shown below:

1. *POST /api/v4/groups/import*
2. *POST /api/v4/groups/import/authorize*
3. *POST /api/v4/groups*
4. *GET /api/v4/groups*
5. ...

To improve reproducibility, we provide the fixed prompts used for LLM-assisted Sequence Annotation and Endpoint Semantic Categorization in Appendix A. The prompts constrain the LLM to a predefined category set and require valid JSON output, making the annotation process a closed-set classification task rather than open-ended generation.

3.3. Sequence Category Prediction Model

3.3.1. Objective and Formulation

We build a sequence category prediction model to estimate the most likely API category of the next step given an already executed attack prefix. Figure 2 illustrates an example of a three-step fuzz-sequence construction process. The model provides a data-driven mechanism for category guidance and ranking during fuzz-sequence construction. Unlike traditional endpoint-level prediction, this formulation explicitly leverages the executed multi-step interaction prefix together with response feedback (i.e., status codes) to predict the next category in the attack chain.

Given an executed attack sequence prefix:

$$X_{1:t} = \{(c_1, s_1, r_1), \ldots, (c_t, s_t, r_t)\}$$

where C_i denotes the category of step i, S_i denotes the status-code class (2xx/3xx/4xx/5xx/other), and r_i denotes the exact response code (e.g., 200, 500, or unknown); the model outputs the conditional distribution over the next-step category:

$$P(c_{t+1} \mid X_{1:t}, \text{target}, \text{service})$$

In implementation, training samples are constructed using a sliding-window strategy: for each position t in a sequence, the prefix $X_{1:t}$ is used as the input and C_{t+1} is used as the supervision label, yielding a large number of prefix-to-next-category training pairs. We further introduce a 5xx-trigger bias weight to emphasize anomaly related response categories, allowing the model to favor high-risk paths during fuzz guidance.

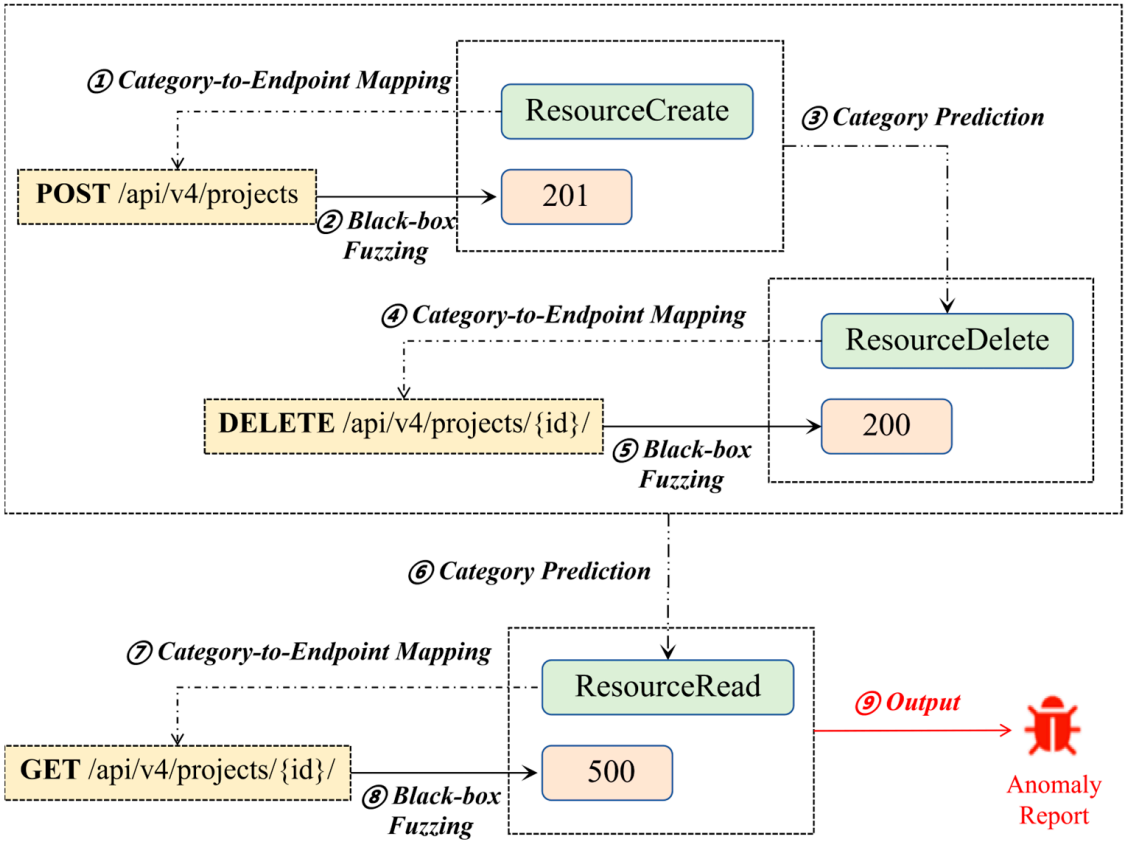


Figure 2. Example of a three-step fuzz-sequence construction process.

3.3.2. Model and Training Objective

The model is implemented as a conditional causal Transformer encoder [37]. At each time step, the input representation is formed by summing multiple embeddings, including category embedding, status-class embedding, exact-response-code embedding, and positional embedding, together with global condition embeddings for the target and the optional service. Specifically,

$$x_i = e_{c_i}^{\text{cat}} + e_{s_i}^{\text{scls}} + e_{r_i}^{\text{rcode}} + e_i^{\text{pos}} + e^{\text{tgt}} + e^{\text{svc}}$$

The encoder adopts a causal mask to preserve autoregressive semantics, i.e., the representation at step i can only access historical information $\leq i$. The hidden state h_t at the last valid position is fed into a linear classification head to produce logits for the next-step category:

$$\hat{y} = Wh_t + b$$

The training objective is a weighted cross-entropy loss that assigns higher weights to samples whose target response class is 5xx, thereby increasing the model's sensitivity to error-triggering paths. The loss is defined as follows:

$$\mathcal{L} = \frac{\sum_{i=1}^{N} w_i \cdot \text{CE}(\hat{y}_i, y_i)}{\sum_{i=1}^{N} w_i}, \qquad w_i = \begin{cases} \lambda, & \text{if } s_{t+1} = 5xx, \\ 1, & \text{otherwise} \end{cases}$$

where lambda is the 5xx weighting coefficient (5.0 or 8.0 by default). We train the model with AdamW [38], support early stopping, and report Top-1 and Top-3 accuracy on the validation set to measure the quality of category guidance.

Implementation details. The sequence prediction model is implemented as a conditional causal Transformer encoder. In our prototype, the maximum context length is 32, the embedding dimension is 256, the number of attention heads is eight, the number of Transformer layers is four, and the dropout rate is 0.1. The input representation is the sum of category, status-class, exact-status-code, positional, target, and optional service embeddings. A causal mask is used in the Transformer encoder, and the hidden state at the last valid position is fed into a linear head to predict the next API category.

The model is trained with AdamW for at most 30 epochs, using a learning rate of 2×10^{-4}, a weight decay of 1×10^{-2}, a batch size of 128, label smoothing of 0.05, gradient clipping with maximum norm 1.0, and early stopping patience of four. The train/validation split ratio is 80%/20%, with random seed 42. During training, exact status-code augmentation is applied with probability 0.05, and samples whose label step corresponds to a 5xx status class are assigned a loss weight of 8.0. The validation loss is computed without the 5xx weighting bias.

3.3.3. Inference for Fuzz Guidance

During inference, the model takes the currently executed category sequence and response-code sequence as input, encodes them using the same procedure as in training, and outputs the Top-k candidate categories together with their probabilities. The logits are normalized with softmax, and the k highest-probability categories are selected to prioritize DFS state expansion and sequence construction.

It should be noted that the prediction model is used as a guidance component rather than an error oracle. Top-1 and Top-3 accuracies are reported only to measure how well the model recovers the next API category in held-out historical PoC sequences. Specifically, Top-1 accuracy means that the highest-probability predicted category is the same as the ground-truth next category, while Top-3 accuracy means that the ground-truth next category appears among the three highest-probability predictions. These metrics indicate whether the model has learned useful category-transition patterns from previous PoCs, but they do not directly measure the final reproducible error discovery capability. To avoid data leakage, the train/validation split is performed at the original PoC level before applying the sliding-window strategy. Thus, all prefix-to-next-category samples generated from the same PoC are assigned to the same split, instead of being randomly divided after windowing. This prevents highly similar prefixes from the same attack chain from appearing in both training and validation sets. In addition, the GitLab and WordPress error-triggering sequences reported in our evaluation are not used for training the prediction model. Therefore, the online fuzzing results on these targets provide an initial indication that category-transition patterns learned from public PoCs can guide exploration on unseen REST API systems. However, we do not claim that the current evaluation constitutes a comprehensive cross-project transfer study. In online fuzzing, real errors may not strictly follow historical patterns, and the predicted categories are used only as exploration priors to reduce blind endpoint-level sequence enumeration. The final discovery of reproducible errors still depends on endpoint mapping, payload availability, black-box fuzzing, response feedback, and replay-based validation.

3.4. Online Stateful Exploration

The multi-step stateful DFS (Depth-First Search) fuzzing procedure in Spec2SeqFuzz consists of three tightly coupled components: the DFS scheduling mechanism [39], the category-guided expansion strategy, and the virtual-machine snapshot rollback workflow. Figure 3 shows an illustrative example.

During execution, the system first applies a black-box fuzzer to seed endpoints, collects reusable payloads and execution states, and decides whether a path should continue to expand based on the observed status codes. It then queries the sequence category prediction model for Top-k candidate categories, which narrows the search space and prioritizes high-risk paths. Through category-to-endpoint mapping, the system selects concrete endpoints as the next nodes on the path. This process repeats until the path terminates, after which the search enters the backtracking stage. During backtracking, virtual-machine snapshots restore the system to a consistent baseline state, enabling repeatable experiments and preserving reachability under multi-step dependency constraints.

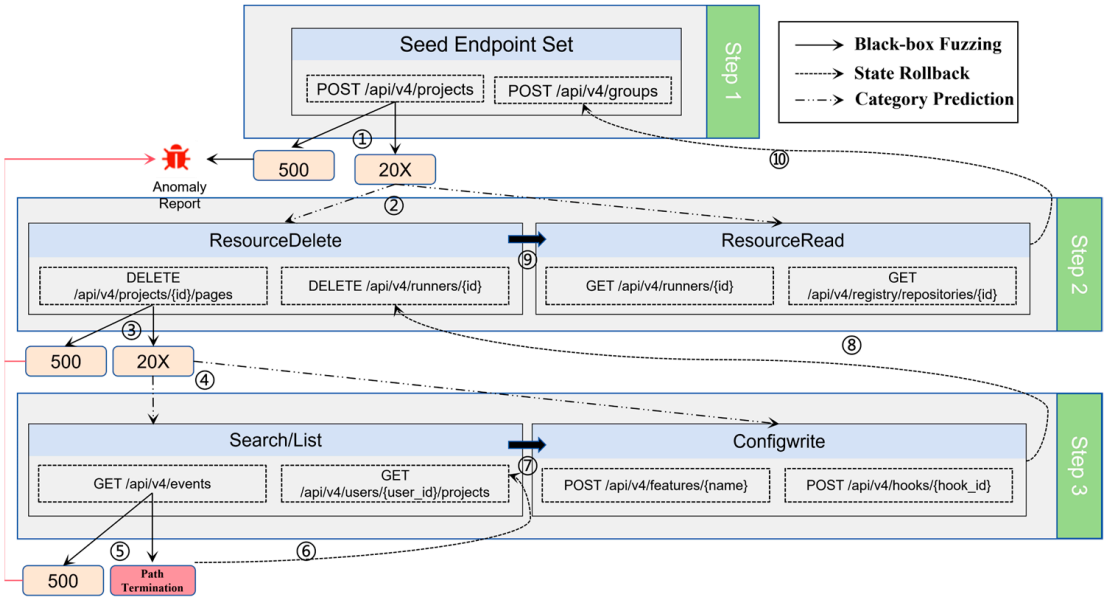


Figure 3. Example workflow of multi-step stateful DFS fuzzing in Spec2SeqFuzz.

3.4.1. DFS Scheduling and Category Guidance

Under DFS scheduling, the system treats the currently executed sequence prefix as the search state, which includes the executed endpoint sequence, the response-status sequence, and related context such as payloads and known parameter values. At each expansion depth, the system first queries the sequence prediction model for the Top-k categories of the next step. It then retrieves the corresponding endpoint candidates from the category-to-endpoint mapping and constructs candidate expansions. For each candidate endpoint, the system invokes black-box fuzzing and records reusable payloads for subsequent parameter filling and path unlocking. This procedure is implemented through seed parsing, category mapping, and layer-by-layer expansion, including endpoint normalization, reverse indexing from categories to endpoints, and Top-k category configuration that can be adjusted by depth.

Algorithmically, the DFS scheduler can be abstracted as follows. Given a sequence prefix S = (e_1,...,e_t), the system first obtains the next-step category set C_{t+1}. For each $c \in C_{t+1}$, it expands the corresponding candidate endpoint set E_c. For each $e \in E_c$, the system executes fuzzing; if the execution succeeds or yields critical payloads, the resulting state is pushed onto the stack for further exploration. This strategy preserves depth-first coverage while reducing ineffective expansions through category guidance, and is particularly effective for reaching deep paths in the presence of multi-step dependencies. Algorithm 1 presents the category-guided stateful DFS fuzzing procedure.

Algorithm 1: Category-Guided Stateful DFS Fuzzing

Input: Seed endpoints S_0, category map M (endpoint → category), model P, max steps K
Output: Discovered issues and payloads

```
C ← INVERT(M); // category → endpoints
stack ← [(s, [], []) | s ∈ S_0]; // (current endpoint,
cat_seq, status_seq)
while stack ≠ ∅ do
    (e, cat_seq, status_seq) ← stack.Pop();
    Execute fuzzing on endpoint e, collect status and
    payloads;
    Append CATEGORY(e) and status to cat_seq and
    status_seq;
    if |cat_seq| ≥ K then
        continue;
    TopCats ← P.PREDICT(cat_seq, status_seq, topk);
    foreach cat ∈ TopCats do
        foreach e' ∈ C[cat] do
            if not ADMISSIBLE(e', payloads) then
                continue;
            stack.PUSH((e', cat_seq, status_seq));
```

Here, $ADMISSIBLE(e', payloads)$ denotes executability checking based on the collected payloads, path parameters, and OpenAPI-required parameters. The system uses payloads produced during fuzzing—including key fields returned by successful 2xx responses—to instantiate the parameters required by subsequent endpoints, thereby ensuring reachability and state consistency during DFS expansion.

3.4.2. VM Snapshot Rollback

To execute multi-step attack chains reproducibly across different branches, Spec2SeqFuzz introduces a rollback mechanism based on virtual-machine snapshots. The purpose of this mechanism is not merely convenience; rather, it provides state isolation for branch exploration. In stateful REST API fuzzing, one branch may create resources, change permissions, update configurations, or leave persistent artifacts that directly affect the behavior of later branches. Without rollback, the search quickly becomes sensitive to branch ordering and accumulated side effects.

Our workflow uses a baseline snapshot that represents a clean initial state of the target system. Before exploring a new top-level branch, the framework restores this baseline snapshot. During depth-first exploration, the system may additionally create intermediate snapshots after prefixes that successfully establish reusable state, such as object creation or other persistent 2xx outcomes. When a sibling branch must be explored, the framework restores the deepest snapshot consistent with the preserved prefix instead of reinitializing the whole environment from scratch. This design reduces re-execution cost while keeping branch semantics reproducible.

More concretely, snapshot management is tied to the DFS stack. When a prefix is extended and the new step does not create useful persistent state, the system continues without introducing a new checkpoint. When a step returns a successful response that materially changes the reachable state space, the framework may checkpoint the environment at that depth so that descendant branches can reuse the established state. When backtracking occurs, deeper temporary snapshots are discarded and the system returns

to the appropriate parent snapshot. In this way, rollback is integrated into the exploration logic rather than treated as an external experiment-management step.

This mechanism offers two advantages. First, it reduces state contamination and improves the reproducibility of reported error-triggering sequences. Second, it makes deep exploration more practical by avoiding full environment reconstruction for every sibling branch. We do not claim that the current paper provides a comprehensive measurement of snapshot overhead; instead, the snapshot design should be understood as an engineering mechanism that stabilizes multi-step exploration in the current prototype.

3.4.3. Black-Box Fuzzer Collaboration and Payload Collection

Spec2SeqFuzz is designed to work with existing black-box API fuzzers rather than replace them. In our prototype, black-box fuzzers are responsible for endpoint-level input generation and local anomaly triggering, while the upper-layer scheduler decides which category and endpoint to explore next. A unified proxy layer connects these two levels by collecting request/response traffic, extracting reusable runtime artifacts, and producing replayable failure reports.

The payload-collection mechanism is centered on successful 2xx responses, because these responses are the main source of runtime values required by later requests. During execution, the proxy parses response bodies and selected response metadata, and retains fields that are likely to be reused across steps, such as identifiers, handles, slugs, tokens, keys, or other application-specific object references. The collected values are stored together with the logical endpoint and response context that produced them, forming a payload repository that accumulates reusable runtime artifacts along the executed prefix.

Producer–consumer alignment is then performed in a spec-constrained manner. For a candidate next endpoint, the scheduler checks whether its required path, query, or body parameters can be instantiated using values available in the payload repository of ancestor steps. In other words, a later endpoint is considered admissible only when the currently executed prefix has already produced values that satisfy the endpoint's required placeholders or required fields. This alignment is therefore not based on arbitrary string matching alone; it is constrained by the parameter structure described in the OpenAPI document and by the runtime values actually observed in prior successful executions.

This design clarifies the role of payload reuse in our framework. We do not claim that reusing values across requests is new by itself; prior API testing systems have already shown the importance of cross-request data flow. Our contribution is to integrate payload reuse tightly with category-guided online exploration and black-box endpoint fuzzing. The proxy layer turns endpoint-level fuzzing outputs into reusable assets for later steps, thereby connecting local input exploration with global multi-step reachability.

In addition to successful responses, the proxy also records abnormal responses such as 5xx failures together with the corresponding requests and contextual prefix information. These artifacts support later replay, triage, and minimization. To reduce noise and avoid unnecessary exposure of sensitive information, the framework filters collected payloads and masks sensitive headers in generated reports.

4. Evaluation

4.1. Experiment Setup

To evaluate the current prototype of Spec2SeqFuzz, we study its error-discovery capability on real REST API systems from three aspects: targets, baseline, and execution environment. Our current evaluation is designed to answer the following question: on two large and practically relevant REST API systems, can category-guided stateful fuzzing

help discover more reproducible error-triggering sequences than a reproduced multi-step REST API fuzzing baseline?

Benchmarks. Following the target choice reported in MINER, we evaluate GitLab and WordPress [40,41]. These two systems are widely used, open-source, and API-rich, and they expose diverse state transitions such as resource creation, update, deletion, content management, and permission-relevant operations. Their large endpoint spaces make them suitable stress tests for stateful sequence construction. Table 2 summarizes the detailed information about the targets.

Table 2. Detailed information about the test targets.

Target	Number of APIs	API Count by HTTP Method	Number of Seeds	Fuzzing Timeout
GitLab 14.1.0.ce.0.	995	DELETE: 112 GET: 445 HEAD: 3 PATCH: 6 POST: 228 PUT: 201	48	12 h
WordPress 5.8.1	160	DELETE: 18 GET: 77 PATCH: 15 POST: 35 PUT: 15	12	12 h

We do not include Bugzilla in the current comparison [42] because MINER reported limited representative multi-step findings on that target. More importantly, the present version of our evaluation is intentionally narrow in scope: we focus on two complex systems that allow us to study deep multi-step exploration in detail. This choice should be interpreted as an initial empirical study rather than a comprehensive benchmark over the broader REST API testing landscape.

Baseline. We use MINER as the primary reproduced baseline in the current paper. MINER is directly relevant because it explicitly targets REST API fuzzing and multi-step sequence construction. At the same time, we do not claim that MINER alone fully represents the current state of the art. Recent systems are also highly relevant baselines for future comparison. For example, EvoMaster represents an important OpenAPI-based automated testing system [18], while DeepRest studies learning-based REST API test generation [29]. LlamaRestTest and AutoRestTest further explore language-model-assisted REST API testing and exploration [30,31]. We therefore frame the present results as a focused comparison with a reproduced multi-step baseline rather than a definitive ranking against all recent approaches.

Reported outcomes. Following MINER, we primarily report errors triggered during REST API fuzzing rather than treating every abnormal behavior as a confirmed vulnerability. In particular, a generated request or request sequence is considered to trigger an error when it causes an abnormal server response, such as a response in the 50× range, or violating a predefined security rule. We further distinguish unique errors and reproducible errors. Unique errors are first grouped according to the triggered endpoint, response status, and request-sequence pattern. Reproducible errors are then obtained after replay-based validation and manual deduplication, following the analysis practice used in MINER. In this paper, we report two types of reproducible errors: (1) multi-step errors that require interactions across multiple endpoints, and (2) single-endpoint errors that can be reproduced through one endpoint alone. We do not claim that every reproducible error is a previously

unknown security vulnerability, a vendor-confirmed bug, or a unique root cause. Some cases may correspond to generic server errors, known issues, duplicates, or configuration-dependent failures. A finer-grained security triage, including distinguishing confirmed vulnerabilities from non-security robustness issues, requires additional manual root-cause analysis and responsible disclosure, and is beyond the scope of the current study.

Environment. All experiments are conducted on a host machine equipped with an AMD Ryzen 9 7950X processor and 32 GB of memory. The target applications and fuzzing workflow run inside a virtual machine so that snapshot rollback can be used to reduce cross-branch state contamination and improve reproducibility.

Scope of interpretation. Because the current study uses two targets and does not include repeated randomized trials or statistical testing yet, the conclusions in this section should be interpreted with appropriate caution. Our goal here is to show the practicality and promise of category-guided stateful exploration, while broader empirical validation remains future work.

4.2. RQ1: Fuzzing Performance Analysis

We evaluate fuzzing performance from the perspective of reproducible errors, following the error-reporting terminology used in MINER. In particular, we separately count the following: [6] multi-step cases that require interactions across multiple endpoints to be triggered, and [42] cases that can be triggered by a single endpoint alone. In this paper, a reported reproducible error refers to an error-triggering request or request sequence that can be replayed under our validation workflow and still triggers the same abnormal server behavior. This term is used to distinguish reproducible errors from transient fuzzing anomalies, not to claim that every case is a newly discovered security vulnerability. We use this term to distinguish validated findings from transient fuzzing anomalies, not to claim that every case is necessarily a previously unknown vulnerability in the target project.

Table 3 summarizes the multi-step error-triggering sequences discovered on GitLab and WordPress. Spec2SeqFuzz finds eight reproducible multi-step errors on GitLab, compared to five found by MINER; on WordPress, the two systems discover four and one reproducible multi-step errors, respectively. For reproducible single-endpoint errors, Spec2SeqFuzz finds 15 on GitLab, whereas MINER finds three; on WordPress, Spec2SeqFuzz finds 12, while MINER finds one. These results suggest that Spec2SeqFuzz not only reaches deep state-dependent errors more frequently on these two targets, but also achieves stronger endpoint-level error-triggering coverage in the current study.

The above advantage can be explained by the difference in exploration mechanisms. MINER mainly relies on dependency inference and valid-sequence-template-driven exploration. When error triggering requires category-crossing state transitions—e.g., creating an object and then entering subsystems such as access control, package management, or branch protection—and when the key preconditions cannot be stably reconstructed through local dependency inference alone, its search is more likely to linger around sequences that are executable but weak in state progression. In contrast, Spec2SeqFuzz maps sequence exploration into a coarser-grained and more transferable category space, and uses sequence category prediction to output the top-k next-step categories. Under the same budget, this significantly reduces the branching factor and concentrates execution on paths that are more likely to yield meaningful state transitions, enabling the discovery of three-step triggering sequences not covered by MINER.

The advantage of Spec2SeqFuzz further stems from the closed loop between sequence category prediction and the black-box fuzzer collaboration. Figure 4 compares the number of reproducible single-endpoint errors found by the two systems. MINER discovers three and one such errors on GitLab and WordPress, respectively, whereas Spec2SeqFuzz

discovers 15 and 12. This is because, after mapping category candidates back to concrete endpoints, our system invokes mature black-box fuzzers for input generation and coverage exploration. Meanwhile, the proxy layer extracts key fields from responses and stores them in a payload repository for subsequent parameter instantiation and path unlocking, such as object identifiers and privilege bindings. As a result, Spec2SeqFuzz can construct valid prefixes that satisfy necessary preconditions with higher probability, and can continuously advance toward deeper state-dependent endpoints, thereby improving both multi-step and single-endpoint error discovery.

Table 3. Reproducible multi-step errors discovered by MINER and Spec2SeqFuzz on GitLab and WordPress.

Target	No.	Endpoint	MINER	Spec2SeqFuzz
GitLab	1	1. POST/api/v4/projects 2. POST/api/v4/projects/{id}/fork	√	√
	2	1. POST/api/v4/projects 2. PUT/api/v4/projects/{id}	√	√
	3	1. POST/api/v4/projects 2. GET/api/v4/projects/{id}/ci/lint		√
	4	1. POST/api/v4/projects 2. DELETE/api/v4/groups/{id} 3. GET/api/v4/projects/{id}/custom_attributes	√	√
	5	1. POST/projects/{id}/issues 2. PUT/projects/{id}/issues/{issue_id}	√	√
	6	1. POST/api/v4/groups 2. DELETE/api/v4/groups/{id} 3. GET/api/v4/groups/{id}	√	√
	7	1. POST/api/v4/projects 2. GET/api/v4/projects/{id}/pages_access 3. GET/api/v4/projects/{id}/packages/nuget/index		√
	8	1. POST/api/v4/projects 2. GET/api/v4/projects/{id}/statistics 3. GET/api/v4/projects/{id}/protected_branches/{name}		√
WordPress	1	1. POST/categories 2. DELETE/categories/{id} 3. POST/categories	√	√
	2	1. POST/wp/v2/categories 2. POST/wp/v2/posts/{id}/autosaves		√
	3	1. POST/wp/v2/posts 2. PUT/wp/v2/posts/{id}		√
	4	1. POST/wp/v2/pages 2. PUT/wp/v2/pages/{id}		√

4.3. RQ2: Ablation Study

To better understand the contribution of the key components in Spec2SeqFuzz, we conduct ablation studies under the same target systems and experimental environment described in Section 4.1. All experiments are run under a fixed fuzzing budget of 12 h, and we separately count the following: (1) reproducible multi-step errors that require interactions across multiple endpoints to be triggered, and (2) reproducible single-endpoint errors that can be triggered by a single endpoint alone.

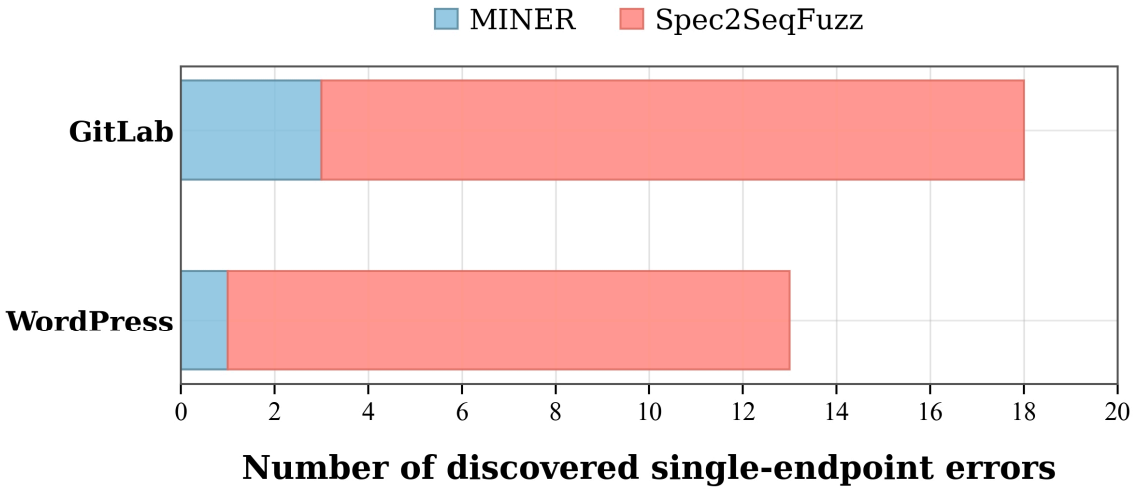


Figure 4. Comparison of reproducible single-endpoint errors found by MINER and Spec2SeqFuzz on GitLab and WordPress.

We consider the following four configurations:

Spec2SeqFuzz: The full system, including sequence category prediction, payload collection and reuse, and DFS-based stateful exploration.

w/o_SeqPredict: This replaces the sequence category prediction module with random sequence generation. At each step, the system no longer predicts the next category based on the executed prefix, but randomly selects endpoints to extend the sequence.

w/o_PayloadCollect: This disables payload collection and reuse; therefore it no longer extracts parameters from successful responses for use in later steps.

w/o_BFS: This replaces the original DFS-based stateful exploration with BFS (breadth-first search), while keeping the remaining components unchanged.

Table 4 shows that the full Spec2SeqFuzz substantially outperforms all three ablated variants in reproducible multi-step error discovery. Specifically, the full system finds 12 reproducible multi-step errors across GitLab and WordPress, including eight on GitLab and four on WordPress. By contrast, w/o_SeqPredict finds only three, w/o_BFS also finds only three, and w/o_PayloadCollect finds no reproducible multi-step errors at all. In other words, removing sequence category prediction or replacing DFS with BFS reduces the number of discovered multi-step errors to one quarter of that of the full system, while removing payload collection and reuse almost completely eliminates multi-step error-discovery capability.

More importantly, this difference is not reflected only in the total number of errors, but also in the depth of the reachable errors. According to Table 4, the full system discovers 12 multi-step errors in total, including seven two-step errors and five three-step errors. In contrast, all three errors found by w/o_SeqPredict and w/o_BFS are two-step errors; neither variant discovers any three-step error. This result shows that the advantage of the full system lies not merely in finding more errors, but in reaching deeper error-triggering paths that depend more heavily on state progression.

Contribution of sequence category prediction. We first examine the role of sequence category prediction. Replacing this module with random sequence generation reduces the number of discovered reproducible multi-step errors from 12 to 3. On GitLab, the number drops from eight to two; on WordPress, it drops from four to one. Moreover, this variant discovers only a few two-step errors and fails to explore any three-step error.

This result suggests that, in stateful multi-step API fuzzing, randomly enumerating endpoint combinations is insufficient for approximating the preconditions required by reproducible errors. The reason is fundamental: multi-step errors are rarely triggered by

arbitrary endpoint concatenation, but instead depend on semantically meaningful state-transition chains. For example, an earlier step may need to create a referenceable object, or produce identifiers, privilege relations, or configuration state required by later requests. Only when the semantics of such a prefix are satisfied can subsequent endpoints reach an anomalous state. Random sequence generation lacks this directionality, and thus wastes substantial budget on paths that are either executable but unproductive, or incapable of forming meaningful state progression at all.

Table 4. Reproducible multi-step errors discovered by Spec2SeqFuzz and its ablated variants.

Target	No.	Endpoint	Spec2Seq Fuzz	w/o _SeqPredict	w/o _PayloadCollect	w/o_BFS
GitLab	1	1. POST/api/v4/projects 2. POST/api/v4/projects/{id}/fork	√	√		√
	2	1. POST/api/v4/projects 2. PUT/api/v4/projects/{id}	√			√
	3	1. POST/api/v4/projects 2. GET/api/v4/projects/{id}/ci/lint	√			
	4	1. POST/api/v4/projects 2. DELETE/api/v4/groups/{id} 3. GET/api/v4/projects/{id}/custom_attributes	√			
	5	1. POST/projects/{id}/issues 2. PUT/projects/{id}/issues/{issue_id}	√	√		
	6	1. POST/api/v4/groups 2. DELETE/api/v4/groups/{id} 3. GET/api/v4/groups/{id}	√			
	7	1. POST/api/v4/projects 2. GET/api/v4/projects/{id}/pages_access 3. GET/api/v4/projects/{id}/packages/nuget/index	√			
	8	1. POST/api/v4/projects 2. GET/api/v4/projects/{id}/statistics 3. GET/api/v4/projects/{id}/protected_branches/{name}	√			
WordPress	1	1. POST/categories 2. DELETE/categories/{id} 3. POST/categories	√			
	2	1. POST/wp/v2/categories 2. POST/wp/v2/posts/{id}/autosaves	√			
	3	1. POST/wp/v2/posts 2. PUT/wp/v2/posts/{id}	√	√		√
	4	1. POST/wp/v2/pages 2. PUT/wp/v2/pages/{id}	√			

By contrast, the full Spec2SeqFuzz maps multi-step exploration into a coarser-grained and transferable category space, and predicts the top-k next-step categories based on the category and status-code sequence of the executed prefix. This turns search from blind endpoint-level concatenation into guided category-level expansion, substantially reducing invalid branches. The fact that the full system discovers several three-step errors while the random variant discovers none directly shows that sequence category prediction is the key mechanism that allows the search to move beyond shallow states and enter deeper error-triggering regions.

Contribution of payload collection and reuse. Among the three ablated components, payload collection has the most significant impact. Table 3 shows that w/o_PayloadCollect finds no reproducible multi-step error at all, dropping from 12 in the full system to zero. This result indicates that, for multi-step reproducible error discovery, sequence structure alone is not sufficient: the system must also carry concrete data dependencies produced by earlier steps into subsequent requests. Many reproducible multi-step errors depend on dynamically generated runtime parameters, such as object IDs, path handles, category IDs,

article or page identifiers, privilege bindings, or other fields returned by successful server responses. Even if the system can semantically predict which category of endpoint should be visited next, the subsequent request still cannot be instantiated correctly unless such fields are extracted from previous responses and reused. The zero multi-step errors found by this ablated variant directly reflect this issue. This result, in turn, highlights the necessity of our design that combines black-box fuzzer collaboration with proxy-based payload collection. Black-box fuzzers are effective at generating diverse inputs for individual endpoints, but their native capabilities do not automatically maintain data dependencies across requests. Spec2SeqFuzz addresses this gap by extracting reusable fields from 2xx responses through a unified proxy layer and storing them in a payload repository, thereby transforming the local coverage benefits of black-box fuzzing into parameter assets usable by subsequent steps. In other words, payload collection is not an optional optimization; it is a basic condition for making multi-step stateful fuzzing work.

Contribution of DFS scheduling. We further compare the impact of the search strategy. Replacing DFS with BFS reduces the number of discovered reproducible multi-step errors from 12 to three—specifically, from eight to two on GitLab and from four to one on WordPress. Similarly to the random sequence variant, all three errors found by w/o_BFS are two-step errors, and none is a three-step error.

We clarify that this result should not be interpreted as a general claim that DFS is universally superior to BFS for all REST API fuzzing scenarios. Instead, our conclusion is limited to the fixed-budget setting of Spec2SeqFuzz: under the same 12 h fuzzing budget and with the same category predictor, payload reuse mechanism, black-box fuzzer, authentication setting, target environment, and snapshot workflow, DFS achieves better end-to-end error-discovery capability than BFS. In this paper, the primary goal is to evaluate a complete REST API testing system, and therefore the number of reproducible errors discovered within a fixed time budget is the most direct effectiveness metric.

We agree that path coverage, valid-sequence ratio, average depth, and executions per unit time are useful intermediate metrics for understanding the internal behavior of different search strategies. However, these metrics do not directly determine whether a testing system can finally expose reproducible failures. For example, a strategy may cover more shallow paths or execute more requests per unit time, but still fail to reach the state-dependent conditions required by deep multi-step errors. In contrast, the final number of reproducible multi-step errors reflects the practical objective of our system more directly.

The reason DFS performs better in our setting is that once a prefix establishes useful runtime state and reusable payloads, DFS continues to extend this prefix and can more fully exploit the accumulated object states, identifiers, and contextual information. BFS, by contrast, tends to allocate a large portion of the fixed budget to expanding many shallow prefixes. Although this may provide broader layer-wise exploration, it can prevent promising prefixes from being pushed deeper before the budget is exhausted. Therefore, in the current Spec2SeqFuzz workflow, DFS improves practical error-discovery capability by better preserving and exploiting stateful context along deep paths.

This result shows that, under a fixed budget, search order itself is a first-order factor that affects deep-error discovery. BFS tends to expand many candidate branches at shallow depths, attempting to enumerate a large number of short prefixes. However, for complex systems described by OpenAPI, the space of endpoint combinations is enormous, and breadth-first exploration consumes substantial budget at shallow layers. As a result, many potentially promising prefixes cannot be pushed deeper before the budget is exhausted. Although BFS offers better theoretical layer coverage, in practical multi-step error discovery, it is more likely to remain trapped in the large-scale enumeration of shallow executable paths, rather than reliably reaching errors that truly depend on multi-step state

progression. Under the fixed-budget setting of Spec2SeqFuzz, DFS demonstrates stronger error-discovery capability than BFS, as measured by the number of reproducible multi-step errors discovered within the same time budget. Once the current prefix has established meaningful state progression, DFS continues to explore downward along that path, making fuller use of the payloads, object states, and contextual information accumulated along the prefix, and thus increasing the probability of reaching deep anomalous states.

Figure 5 compares the number of reproducible single-endpoint errors found by the four systems on GitLab and WordPress. Unlike the multi-step error results in Table 4, the performance gap on single-endpoint errors is relatively small, and the ablated variants do not exhibit degradation as severe as that observed for multi-step errors. This behavior is consistent with the design of our system: single-endpoint error discovery depends primarily on the input generation and anomaly triggering capability of the black-box fuzzer, rather than strongly relying on cross-step state progression, parameter transfer, or search scheduling. The results in Figure 5 validate the effectiveness of black-box fuzzer collaboration from another perspective: even after removing sequence prediction, payload collection, or replacing DFS with BFS, the system can still maintain a non-trivial level of single-endpoint error-discovery capability as long as the black-box fuzzer remains in place. This indicates that the core contribution of Spec2SeqFuzz is not to replace existing black-box fuzzing, but to complement its natural weakness in multi-step state-dependent settings, elevating it from local single-endpoint testing to executable and scalable multi-step error exploration. The relatively small gap in Figure 5 therefore does not weaken the contribution of our method; rather, it reinforces the conclusion that sequence category prediction, payload reuse, and DFS-based stateful scheduling primarily improve the discovery of deep, multi-step, and state-dependent vulnerabilities, which are exactly the weaknesses of traditional black-box API fuzzing.

4.4. RQ3: Hyperparameter Sensitivity

To further understand the behavior of Spec2SeqFuzz under different search configurations, we analyze the impact of two key hyperparameters on error-discovery performance: Max Steps and Max Categories. Max Steps denotes the maximum path depth allowed during stateful multi-step exploration, whereas Max Categories denotes the number of top-k categories output by the sequence category prediction model and actually traversed at each expansion step. These two parameters control the vertical exploration depth and horizontal branching width of the search, respectively, and therefore directly determine how a limited fuzzing budget is allocated between deep path advancement and candidate-endpoint coverage.

Error-discovery results under different Max Steps settings. Figure 6a shows that, on both GitLab and WordPress, the system achieves the best error-discovery performance when Max Steps = 3. This result reflects a typical budget–allocation tradeoff in stateful multi-step fuzzing. When Max Steps = 2, the system can execute more short paths within a fixed time budget, thereby covering a broader set of endpoints and exploring a larger number of paths. However, because the search depth is capped, the system cannot reach deeper error-triggering states that inherently require three-step state progression. In other words, the system becomes biased toward broad coverage, which is suitable for discovering single-endpoint errors or shallow two-step errors, but insufficient for more complex state-dependent scenarios.

When Max Steps increases to three, the system can continue extending beyond two-step prefixes, thereby gaining the ability to reach deeper state-dependent errors. This point is especially important when combined with the results in Sections 4.2 and 4.3: some of the reproducible multi-step errors discovered in this paper inherently require three-step

chains to become observable. If the search depth is limited to two, such errors are impossible to discover by construction. For this reason, Max Steps = 3 provides the best balance between deep-state reachability and acceptable exploration cost, allowing the system to preserve a reasonable level of endpoint coverage while still exploring sufficiently deep-error-triggering paths, and thus yielding the highest overall number of discovered errors.

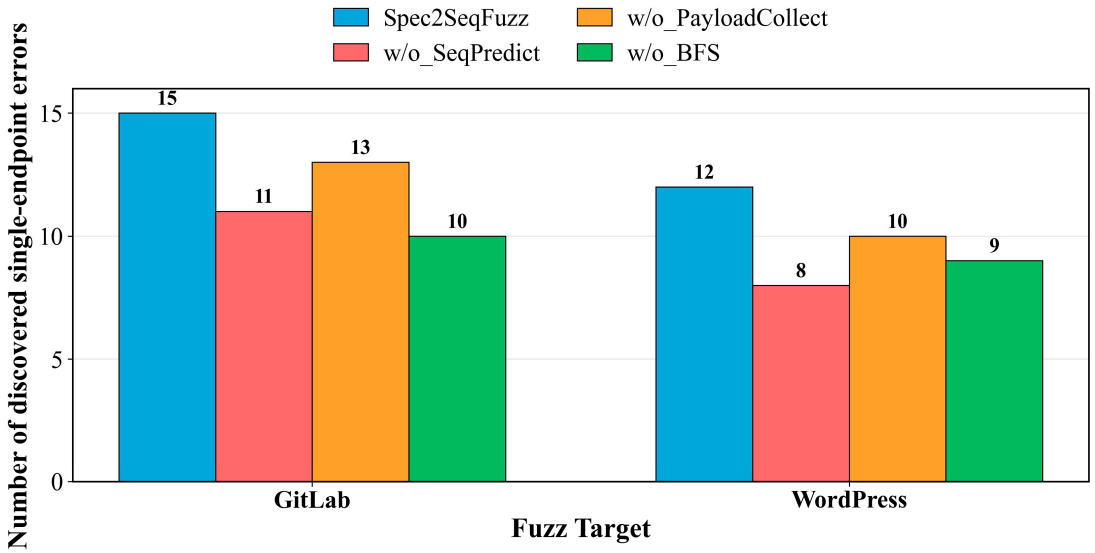
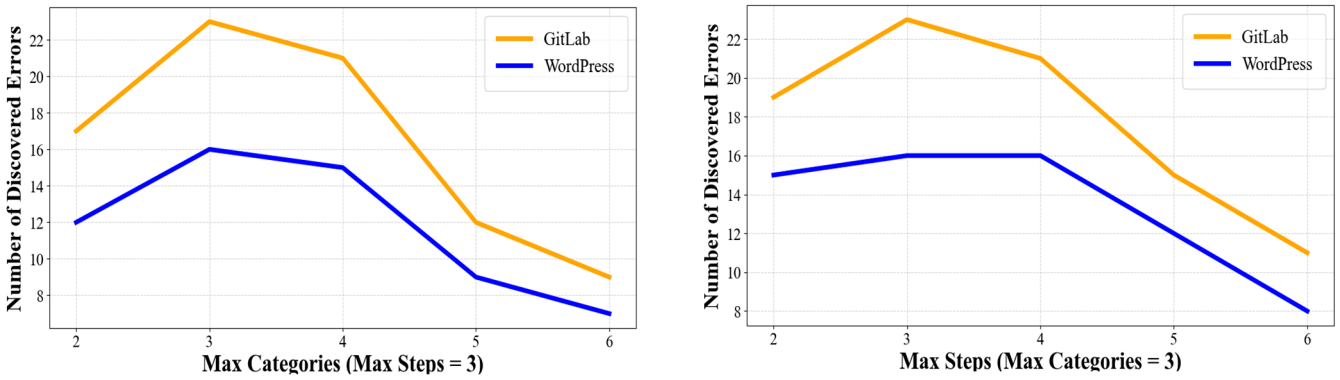


Figure 5. Comparison of reproducible single-endpoint errors found by Spec2SeqFuzz and its ablated variants on GitLab and WordPress.



(**a**) Max Steps (Max Categories = 3) (**b**) Max Categories (Max Steps = 3)

Figure 6. Impact of hyperparameters on error discovery.

When Max Steps are increased further, error-discovery performance begins to decline. The reason is that while greater depth enables more thorough exploration along individual paths, it also significantly increases the execution cost of each path. In our stateful fuzzing workflow, deeper paths require additional rounds of category prediction, category-to-endpoint mapping, black-box execution, payload extraction and reuse, and snapshot rollback. Each additional layer introduces non-trivial time overhead. Under a fixed budget, the total number of paths that can be attempted therefore decreases, and the number of candidate endpoints and prefixes that can be covered decreases accordingly, ultimately leading to fewer discovered errors overall. In other words, an overly large path-depth limit shifts the system from being able to discover deeper errors to over-investing the budget in a small number of paths.

These results suggest that the optimal value of Max Steps is not simply the largest possible one, but rather the point that balances the state depth required by reproducible

errors against the execution cost per path. For the two target systems studied in this paper, three steps are already sufficient to cover the main deep-error patterns, and increasing the depth further does not yield gains commensurate with the additional cost. Under our setting, Max Steps = 3 is therefore the most reasonable empirical choice.

Error-discovery results under different Max Categories settings. We fix Max Steps = 3 and analyze the effect of different Max Categories values on error-discovery performance. Figure 6b shows a clear trend: performance remains stable initially and then declines as the number of categories expand at each layer increases. Overall, Max Categories = 3 and Max Categories = 4 achieve the best and closely comparable performance, whereas error-discovery capability degrades substantially when Max Categories > 4.

When Max Categories = 2, the system retains only a small number of top-k predicted categories at each step, resulting in a relatively small branching factor and enabling the search to advance quickly while covering a large number of endpoints. The advantage of this setting is that the system does not spend too much time expanding candidates at a single layer under a fixed budget, and is therefore relatively favorable for discovering single-endpoint errors or shallow-path errors. As noted in our observations, this setting allows the system to traverse most endpoints and find many errors on single-endpoint or shallow paths, even though it struggles to reach deeper errors. This indicates that a smaller category width improves search efficiency, but also compresses the diversity of deeper paths, making the system more likely to miss complex prefixes that require alternative category combinations.

When Max Categories increases to three or four, the system preserves high search efficiency while also providing a richer set of successor categories for each prefix, thereby balancing effective depth advancement with necessary branching diversity. This behavior is consistent with the role of our sequence category prediction model: the model is not intended to uniquely determine the next step, but rather to provide a set of high-probability candidate categories for DFS to explore further. If too few candidate categories are retained, any local prediction error may lead to premature pruning and the loss of the true error-triggering path. Increasing the width moderately to three or four alleviates this risk without significantly expanding the search space, thereby improving robustness in path expansion. It is therefore consistent with the design of our method of Max Categories = 3 and 4 perform similarly and both outperform smaller or larger settings in Figure 6b.

In contrast, when Max Categories > 4, system performance degrades noticeably. The fundamental reason is that each category often corresponds to multiple concrete endpoints. As the number of retained categories increases, the set of candidate endpoints expands rapidly, causing the cost of each layer to rise substantially. Spec2SeqFuzz does not stop at the category level; instead, it maps categories back to concrete endpoints, invokes black-box fuzzers on them, and collects results through the proxy layer. Consequently, an increase in horizontal width directly amplifies downstream execution cost. Under a fixed time budget, the system then spends too many resources on expanding candidates at individual layers, leaving insufficient budget to explore more paths or push them deeper. As summarized by our observations, when Max Categories exceeds four, the large number of endpoints associated with each retained category prevents the system from exploring enough paths within the allotted time, which in turn leads to a substantial drop in overall error-discovery performance.

4.5. RQ4: Robustness of LLM-Assisted Annotation

Because Spec2SeqFuzz relies on LLMs in the offline stage to assist PoC-step annotation and endpoint categorization, we further evaluate the robustness of this annotation process. The goal of this experiment is not to compare the general reasoning ability of different

LLMs, but to examine whether our annotation task is sensitive to model choice, prompt wording, and model-version differences.

It is important to note that the annotation task in Spec2SeqFuzz is a constrained classification task over a predefined taxonomy, rather than an open-ended generation task. The input is a structured OpenAPI endpoint description or a normalized PoC request step, and the output is restricted to one category label from the taxonomy defined in Section 3.2.1. Therefore, the output space is small and the possibility of unconstrained hallucination is limited.

We randomly sample annotation instances from both OpenAPI endpoint descriptions and normalized PoC request steps. Each sample is independently labeled by several publicly available LLMs, including ChatGPT, DeepSeek-V3, Gemini, and Grok, using the same category taxonomy and annotation instruction. To examine prompt sensitivity, we additionally prepare several semantically equivalent prompt variants that differ only in wording and category-description order. The model outputs are compared with the manually validated final labels used in our experiments.

The results show that the LLM-assisted annotation process is stable across different models and prompt variants. As shown in Table 5, the agreement between each model and the manually validated labels is consistently above 95%, while the annotation error rate remains below 5%. The pairwise agreement among different LLMs and the prompt-level consistency are also above 95%. Most disagreements occur between semantically close categories, such as ResourceRead and Search, or ResourceUpdate and ConfigWrite. These cases are usually caused by endpoints that contain both query and retrieval semantics, or endpoints whose descriptions combine resource modification with configuration update behavior.

Table 5. Robustness of LLM-assisted annotation.

Model	Agreement with Manual Labels	Annotation Error Rate	Prompt Consistency
ChatGPT	98.7%	1.3%	97.1%
DeepSeek-V3	97.6%	2.4%	96.5%
Gemini	98.2%	1.8%	97.8%
Grok	97.1%	2.9%	96.0%

All models are queried with the same taxonomy and prompt template. We use low-temperature or deterministic settings whenever configurable to reduce sampling variance. The final labels used in the experiments are manually validated rather than directly taken from raw LLM outputs.

More importantly, the final labels used by Spec2SeqFuzz are not raw LLM outputs. All LLM-generated labels are manually cross-validated before being used to construct the endpoint-category mapping and the sequence category training data. Therefore, occasional LLM disagreements do not directly enter the online fuzzing stage. Since the online stage only uses the manually validated category mapping and the trained next-category predictor, the small annotation variance observed in this experiment does not materially affect the final fuzzing results.

These results suggest that the LLM-assisted annotation stage is sufficiently stable for the category-guided fuzzing setting studied in this paper. Nevertheless, we acknowledge that more ambiguous APIs or a finer-grained taxonomy may increase annotation difficulty. Future work can further reduce annotation variance through multi-model voting, active learning, or more systematic human-in-the-loop validation.

5. Discussion and Limitation

5.1. PoC-Driven Category Prediction

The sequence category prediction model in Spec2SeqFuzz is built on public PoC data, which introduces a clear source of data dependency. Our current training corpus is primarily derived from publicly collected multi-step PoCs. Such data is attractive because it is easy to obtain, relatively uniform in structure, and amenable to large-scale automated processing. However, it also has important limitations. First, the amount of publicly available data that is truly suitable for learning multi-step stateful attack chains remains limited. Second, community-contributed PoCs vary substantially in completeness, formatting quality, and faithfulness to the real attack logic, making it difficult to ensure that every sample accurately captures the state transitions required to trigger the underlying abnormal behavior. If future work can leverage higher-quality multi-step attack-chain data from enterprise security validation, penetration testing, red-team exercises, or continuous security operations, the category prediction model may learn more stable and fine-grained cross-step dependency patterns, thereby further improving prediction accuracy, cross-system transferability, and the ability to hit deep state-dependent paths.

5.2. Cost of LLM-Assisted Offline Annotation

Spec2SeqFuzz uses LLMs only in the offline stage to preprocess public PoCs, OpenAPI specifications, and endpoint semantics. This design improves reproducibility and execution efficiency during online fuzzing, but it does not eliminate cost; rather, it shifts the cost to the offline knowledge-construction phase. In particular, when a large number of PoC templates, REST API endpoints, and multi-step sequences must be normalized, categorized, and cross-validated, LLM-assisted annotation introduces additional time and monetary overhead. For a single target system or a small-scale study, this cost remains manageable. However, as the target scope expands to more services, richer API error semantics, or a finer-grained category taxonomy, the offline annotation burden grows accordingly. Moreover, the quality of LLM annotations may be influenced by prompt design, model version, and noise in the input data, which introduces another source of instability. An important direction for future work is therefore to design lighter-weight hybrid annotation pipelines that reduce repeated labeling cost during cross-system transfer.

The robustness experiment in Section 4.5 shows that the current annotation task is relatively stable because it is formulated as a closed-set classification problem over structured API descriptions. However, this does not mean that LLM-assisted annotation is error-free. When the taxonomy becomes more fine-grained or when endpoints exhibit highly ambiguous semantics, annotation disagreement may increase. In the current prototype, this risk is mitigated by manual cross-validation before constructing the final category mapping and training set. Future work may further improve this process through multi-model voting and active-learning-based review.

5.3. Beyond "Long" to "Very-Long" Attack Chains

Our current prototype is mainly effective for short-to-medium multi-step chains, especially two- and three-step error-triggering sequences. We acknowledge this limitation explicitly. The reason is twofold. First, our current prediction model is trained on publicly available multi-step PoCs, and the majority of such PoCs contain short attack chains. In particular, two- and three-step chains are much more common than four-step or longer chains in the data we collected. As a result, the model can learn relatively stable category-transition patterns for short-to-medium sequences, but has less training evidence for very long, sparse dependencies. Second, our hyperparameter analysis shows that Max Steps = 3 achieves the best error-discovery capability in the current setting. This does

not mean that longer chains are unimportant. Rather, it indicates that, under a fixed fuzzing budget, increasing the maximum depth introduces substantial additional costs from category prediction, endpoint mapping, black-box fuzzing, payload extraction, parameter instantiation, and snapshot rollback. When the depth becomes too large, the system may spend too much budget on a small number of long paths and therefore cover fewer promising prefixes overall.

Therefore, the goal of this paper is not to prove that the current predictor can strongly model arbitrary-length attack chains. Instead, our goal is to examine whether category prediction can serve as an effective guidance mechanism for stateful REST API sequence construction. In this sense, Spec2SeqFuzz changes the sequence-construction paradigm from blind endpoint-level expansion to category-guided exploration. The evaluation results and ablation study show that this guidance is useful for discovering reproducible multi-step errors under the current experimental setting, even though the current prototype does not fully solve very-long-chain discovery.

We also clarify that the category prediction framework itself is not limited to a single module. Since endpoints are mapped into a shared semantic category space, the model predicts category transitions rather than transitions among concrete endpoints or URL namespaces. Therefore, it can naturally express cross-module transitions, as long as the corresponding endpoints are included in the endpoint-category mapping and the required runtime payloads can be propagated across steps. However, very long and sparse cross-module chains remain challenging because they require more reliable long-range dependency modeling, more long-chain PoC samples, and more aggressive pruning of invalid prefixes.

Future work should therefore focus on collecting more high-quality long-chain PoCs and extending the current predictor with mechanisms better suited for very-long exploration, such as hierarchical category prediction, beam-style prefix retention, payload-aware pruning, and explicit cross-module anchor patterns. These extensions are orthogonal to the current design and can further improve Spec2SeqFuzz's ability to discover longer and more complex state-dependent errors.

5.4. Evaluation Scope and Statistical Validity

The current evaluation has several limitations that should be made explicit. First, the study is conducted on two large open-source targets only. While GitLab and WordPress are practically relevant and endpoint-rich, they do not cover the full diversity of modern REST API ecosystems. Second, the present version of the evaluation focuses on reproduced runs under a fixed setup and does not yet include repeated randomized trials or statistical testing. As a result, the current paper should be read as an initial empirical validation rather than a statistically exhaustive benchmark.

A broader evaluation would ideally include more heterogeneous APIs, additional recent baselines, and repeated runs with appropriate statistical analysis. We leave this expansion to future work. Our current goal is more modest: to evaluate whether the proposed category-guided design is practically useful in complex real-world systems and to analyze the role of its key components through ablation.

The current evaluation does not include repeated randomized trials, confidence intervals, or variance analysis. This is a limitation of the current study. Our experiments are end-to-end system-level fuzzing runs on real REST API systems, where each run requires a 12 h budget and involves target deployment, VM snapshot restoration, black-box fuzzing, proxy-based payload collection, stateful sequence exploration, and replay-based validation. Therefore, a statistically exhaustive repeated-run evaluation over all baselines, ablations, and hyperparameter settings would require substantial additional computational and

manual validation effort. In this paper, we focus on a fixed-budget comparison under the same target environment and configuration, and we validate reported findings through replay to reduce transient noise. The results should therefore be interpreted as initial empirical evidence of the practicality of category-guided stateful exploration rather than as a statistically exhaustive benchmark. Repeated-run evaluation with confidence intervals and variance analysis remains future work.

5.5. Security Impact and Triage Scope

Another limitation concerns security-impact interpretation. In this paper, we focus on validated error-triggering sequences and replayable anomalous behaviors. However, we do not yet perform a full manual triage that classifies every validated case into categories such as newly discovered vulnerability, previously known bug, duplicate report, or non-security robustness issue. Such a distinction is important for deployment-oriented security assessment, but it requires substantial target-specific manual analysis and responsible-disclosure work beyond the current scope.

Accordingly, the contribution of Spec2SeqFuzz should be understood as improving the ability to reach and validate deep anomalous states in REST API systems. A more detailed security-impact study, including root-cause analysis and novelty assessment of each finding, remains future work.

6. Related Work

6.1. Specification-Based and Dependency-Aware REST API Testing

A large body of prior work has studied automated testing and fuzzing for REST APIs from OpenAPI or Swagger specifications. Representative systems extract endpoints, parameters, and dependency information from specifications and then generate executable requests or request sequences. RESTler is a prominent example: it infers producer–consumer dependencies among operations and uses them to drive stateful exploration. MOREST updates service models using execution feedback, while RestTestGen generates tests from operation-dependency relationships. Prior work has also studied automated black-box testing of nominal and error scenarios in RESTful APIs [9,43]. Other approaches focus on resource- and dependency-based test case generation for RESTful web services [11,44]. More recent studies further explore learning-based or model-assisted techniques for improving REST API testing [13,14].

These efforts establish two important points. First, multi-step API testing requires explicit handling of dependencies among operations. Second, cross-request value propagation is fundamental to making later operations executable. Spec2SeqFuzz builds on these insights rather than replacing them. Our difference lies in how the search is organized. Instead of expanding directly in the concrete endpoint or dependency-graph space, we first predict the next step in a compact semantic category space and only then map the predicted category back to candidate endpoints. Thus, our contribution is not the introduction of dependencies or value reuse themselves, but a category-guided exploration mechanism that uses those signals in a different way.

6.2. Vulnerability-Oriented Multi-Step API Security Testing

Another line of work targets reproducible error discovery more explicitly. NAUTILUS shows that many RESTful API vulnerabilities require multiple operations executed in a particular order and therefore cannot be exposed by single-endpoint testing alone [22]. Vulnerability-oriented API testing research has further emphasized the importance of security-oriented operation ordering, precondition satisfaction, and stateful execution.

Spec2SeqFuzz is aligned with this line of work in its focus on deep state-dependent error discovery. Our specific contribution is to combine category-guided sequence planning, proxy-based runtime artifact reuse, black-box fuzzing, and snapshot-assisted rollback in a unified workflow. This design allows the framework to separate three concerns that are often entangled in prior work: semantic sequence guidance, endpoint-level input generation, and cross-branch state management.

6.3. Learning-Based and LLM-Assisted REST API Testing

Machine learning has been widely used to improve fuzzing efficiency [45,46]. In the context of REST API fuzzing, prior work has shown, demonstrating that data-driven techniques can reduce invalid inputs, increase request pass rates, and help testing reach deeper states. More recent work has also introduced LLMs into software testing and fuzzing. For instance, DeepRest explores learning-based REST API test generation [29], while other studies investigate neural or learning-based techniques for improving test generation and fuzzing efficiency [47,48]. LlamaRestTest and AutoRestTest further study language-model-assisted REST API testing and exploration [30,31]. These efforts demonstrate that semantic reasoning can improve REST API testing, especially when surface-level specification information is insufficient.

Spec2SeqFuzz differs from these systems where model intelligence is placed. We do not rely on an LLM online to generate each next request, to reason over each step of the execution, or to remain in the critical path of the fuzzing loop. Instead, we use LLMs only offline to normalize public PoCs, classify endpoints into a reusable category taxonomy, and construct training samples for next-category prediction. Online exploration is then guided by a lightweight learned predictor, black-box endpoint fuzzers, and runtime payload reuse. This design trades off some online flexibility for lower runtime cost, improved reproducibility, and a clearer separation between global sequence planning and local input generation.

6.4. Black-Box API Tools and Tool Synergy

Existing public black-box API testing tools are effective at endpoint-level input generation, protocol-compatible request construction, and anomaly triggering. Because they do not require source code, they are easy to deploy and well suited for quickly exploring local input spaces, making them valuable in practice.

Their strengths, however, lie primarily at the level of individual endpoint execution rather than multi-step sequence organization. Without a higher-level stateful scheduling mechanism, such tools usually test endpoints in isolation and therefore struggle to reach deep state-dependent error states that depend on complex preconditions.

Spec2SeqFuzz is designed to exploit the strengths of black-box fuzzers rather than replace them. In our framework, black-box fuzzers are responsible for endpoint-level input generation and local anomaly triggering; the upper-layer scheduler decides which category and endpoint to explore next; and the proxy layer extracts dynamic fields from responses and stores them as reusable payloads for later steps. In this way, Spec2SeqFuzz decouples global sequence planning from local input generation, allowing it to combine the maturity of existing black-box tools with the global guidance of category-driven exploration.

7. Conclusions

We presented Spec2SeqFuzz, a stateful multi-step fuzzing framework for REST API systems. The framework is built around a simple design principle: guide online exploration in a compact category space, and combine this guidance with endpoint-level black-box fuzzing, runtime payload reuse, and snapshot-assisted rollback. To realize this design, we

use LLMs only in the offline stage to normalize public PoCs, construct a reusable endpoint taxonomy, and prepare training data for next-category prediction.

We implemented a prototype and evaluated it on GitLab and WordPress, using MINER as the primary reproduced baseline in the current study. On these two targets, the results suggest that category-guided stateful exploration is effective for reaching validated multi-step error-triggering cases and also helpful for improving the executability of deeper paths through payload reuse. Our ablation study further indicates that category guidance, payload reuse, and depth-first stateful exploration are complementary components of the overall design.

At the same time, we do not view the current paper as the final word on empirical evaluation. The present study is limited to two targets, one primary reproduced baseline, and a currently incomplete set of intermediate metrics and repeated-run statistics. We therefore interpret the reported results as promising evidence for the practicality of the design, rather than as a definitive state-of-the-art claim. Future work includes broader benchmarking against additional recent tools, repeated statistical evaluation, more detailed measurement of intermediate model and system behavior, and finer-grained security triage of validated findings.

Author Contributions: Conceptualization, Z.H., S.S. and Y.G.; methodology, Z.H., S.S. and Y.G.; software, Z.H.; validation, Z.H.; formal analysis, Z.H.; investigation, Z.H. and A.Z.; writing—original draft, Z.H.; writing—review and editing, Z.H., S.S., Y.G. and A.Z.; visualization, Z.H.; and project administration, Z.H., S.S., Y.G. and A.Z. All authors have read and agreed to the published version of the manuscript.

Funding: The authors gratefully acknowledge the financial support provided by Xiangjiang Laboratory Project (25XJ03004, 25XJ03001, 23XJ01002, 23XJ01007, 24XJ01001); Chinese Academy of Engineering project "Research on the Strategic Layout and Implementation Path of Future Key Industries to Promote the Development of New Quality Productivity" (2025-XBZD-14); the National Natural Science Foundation of China project (72088101); Scientific Research Fund of Hunan Provincial Education Department (24C0307).

Data Availability Statement: The original contributions presented in this study are included in the article. Further inquiries can be directed to the corresponding author.

Conflicts of Interest: The authors declare no conflicts of interest.

Appendix A

Appendix A.1. Prompt for PoC's Sequence Annotation

You are an expert in REST API security testing and multi-step error analysis.

Your task is to annotate each HTTP request step in a normalized multi-step PoC according to a predefined REST API category taxonomy. The goal is to convert the PoC into a category sequence that can be used for training a next-category prediction model for stateful REST API fuzzing.

You must choose exactly one category for each request step from the following category set:

- Auth: login, logout, session creation, token acquisition, session termination, or authentication-related operations.

- UserRegister: user registration, account activation, password reset, or account lifecycle operations.

- ResourceCreate: endpoints that create new referenceable resources or objects.
- ResourceRead: endpoints that retrieve the state or details of a specific resource.
- ResourceUpdate: endpoints that update or modify an existing resource.
- ResourceDelete: endpoints that delete or remove an existing resource.
- Search: endpoints that perform listing, searching, filtering, sorting, pagination, or query-style retrieval.
- FileUpload: endpoints that upload files or import file-like content.
- FileDownload: endpoints that download, export, retrieve, or leak file/data content.
- ConfigWrite: endpoints that modify configuration, administrative settings, permissions, global options, or system-level states.
- BackgroundJob: endpoints that trigger asynchronous jobs, scheduled tasks, build tasks, background execution, or delayed processing.
- HealthCheck: healthcheck, status, metrics, monitoring, or diagnostic endpoints.
- APIGateway: endpoints that proxy, forward, relay, or interact with external services or APIs.
- HiddenAPI: undocumented, hidden, debug, internal, or non-public endpoints.
- Other: endpoints that cannot be reasonably assigned to the above categories.

Annotation rules:
1. Assign exactly one primary category to each request step.
2. If an endpoint has multiple possible meanings, choose the category that best reflects its dominant role in the multi-step attack sequence.
3. Use HTTP method, path, parameters, request body, response expectation, and the role of the step in the PoC to determine the category.
4. For GET endpoints:
- Use Search if the endpoint mainly lists resources, supports filters, pagination, sorting, or keyword queries.
- Use ResourceRead if the endpoint mainly retrieves the state or details of one specific object.
- Use FileDownload if the endpoint retrieves file content, exported data, backups, logs, or downloadable resources.
5. For POST endpoints:
- Use ResourceCreate if the endpoint creates a new object.
- Use Auth if the endpoint creates a session, token, or login state.
- Use FileUpload if the endpoint uploads files or imports file-like data.
- Use BackgroundJob if the endpoint mainly triggers asynchronous execution.
6. For PUT/PATCH endpoints:
- Use ResourceUpdate for ordinary resource modification.
- Use ConfigWrite if the modification affects global, administrative, permission, or system-level configuration.
7. For DELETE endpoints:
- Use ResourceDelete unless the endpoint clearly performs another semantic action.
8. Infer the expected status code if it is explicitly shown in the PoC. If the status code is not explicitly shown, use the most likely successful status-code class for that step, usually 2xx.
9. Do not invent additional steps.
10. Do not output explanations unless requested.
11. Output must be valid JSON only.

Input:

```
{
    "vuln_id": "N/A",
    "vuln_name": "Poc Yaml Weiphp Path Traversal",
    "service": "weiphp",
    "steps": [
      {
        "order": 1,
        "method": "POST",
        "path": "/public/material/_download_imgage",
        "params": {
          "query": {
            "media_id": "1",
            "picUrl": "./../config/database.php"
          },
          "body": "\"1\":1\n"
        }
      },
      {
        "order": 2,
        "method": "GET",
        "path": "/public/home/file/user_pics",
        "params": {}
      },
      {
        "order": 3,
        "method": "GET",
        "path": "/public/uploads/picture/{{img}}",
        "params": {}
      }
    ]
}
```

Output format:

```
{
    "vuln_id": "...",
    "vuln_name": "...",
    "service": "...",
    "cat_seq": ["CategoryOfStep1", "CategoryOfStep2", "..."],
    "status_seq": ["200", "200", "..."],
    "target": "...." #"web" or "api"
}
```

Appendix A.2. Prompt for Endpoint Categorization and Seed Selection

You are an expert in REST API security testing, OpenAPI analysis, and stateful multi-step API fuzzing.

Your task is to analyze REST API endpoints extracted from an OpenAPI specification. For each endpoint, you must complete two tasks:

1. Assign one primary API category according to the predefined category taxonomy.
2. Determine whether the endpoint is suitable as an initial seed endpoint for stateful multi-step fuzzing.

You must choose exactly one category from the following category set:

- Auth: login, logout, session creation, token acquisition, session termination, or authentication-related operations.
- UserRegister: user registration, account activation, password reset, or account lifecycle operations.
- ResourceCreate: endpoints that create new referenceable resources or objects.
- ResourceRead: endpoints that retrieve the state or details of a specific resource.
- ResourceUpdate: endpoints that update or modify an existing resource.
- ResourceDelete: endpoints that delete or remove an existing resource.
- Search: endpoints that perform listing, searching, filtering, sorting, pagination, or query-style retrieval.
- FileUpload: endpoints that upload files or import file-like content.
- FileDownload: endpoints that download, export, retrieve, or leak file/data content.
- ConfigWrite: endpoints that modify configuration, administrative settings, permissions, global options, or system-level states.
- BackgroundJob: endpoints that trigger asynchronous jobs, scheduled tasks, build tasks, background execution, or delayed processing.
- HealthCheck: healthcheck, status, metrics, monitoring, or diagnostic endpoints.
- APIGateway: endpoints that proxy, forward, relay, or interact with external services or APIs.
- HiddenAPI: undocumented, hidden, debug, internal, or non-public endpoints.
- Other: endpoints that cannot be reasonably assigned to the above categories.

Endpoint categorization rules:

1. Assign exactly one primary category to each endpoint.
2. If an endpoint has multiple possible meanings, choose the category that best represents its dominant role in stateful sequence construction and dependency propagation.
3. Use the HTTP method, path, operationId, summary, description, parameters, request-body schema, response schema, and security requirements to determine the category.
4. For GET endpoints:
- Use Search if the endpoint mainly lists resources or supports filters, pagination, sorting, keyword queries, or collection-level retrieval.
- Use ResourceRead if the endpoint mainly retrieves the state or details of one specific resource or object.
- Use FileDownload if the endpoint retrieves files, exported data, logs, backups, archives, attachments, or downloadable content.

- Use HealthCheck if the endpoint mainly returns status, metrics, monitoring, or diagnostic information.

5. For POST endpoints:

- Use ResourceCreate if the endpoint creates a new referenceable resource.

- Use Auth if the endpoint creates a login session, access token, refresh token, or authentication state.

- Use UserRegister if the endpoint registers, activates, resets, or manages an account lifecycle.

- Use FileUpload if the endpoint uploads files or imports file-like content.

- Use BackgroundJob if the endpoint mainly triggers asynchronous execution, scheduled tasks, builds, jobs, or delayed processing.

- Use ConfigWrite if the endpoint modifies administrative, permission, global, or system-level configuration.

6. For PUT or PATCH endpoints:

- Use ResourceUpdate if the endpoint modifies ordinary resource fields.

- Use ConfigWrite if the endpoint modifies global settings, administrative options, permissions, security policies, or system-level configuration.

7. For DELETE endpoints:

- Use ResourceDelete unless another semantic role is clearly dominant.

8. Do not infer categories only from the URL name. Consider the full endpoint semantics, including parameters and request/response schemas.

Seed-endpoint selection rules:

An endpoint should be marked as a seed endpoint if it satisfies at least one of the following conditions:

1. It can be executed without dynamic objects created by previous requests.

2. It requires only authentication, static parameters, or basic request parameters.

3. It can create reusable resources.

4. It can return artifacts required by later endpoints, such as IDs, tokens, handles, slugs, file names, or object references.

5. It is likely to unlock later endpoints in a multi-step sequence.

An endpoint should not be marked as a seed endpoint if it satisfies one of the following conditions:

1. It requires dynamic path parameters such as {id}, {project_id}, {group_id}, {slug}, or {name} that are likely produced by previous requests.

2. It mainly consumes existing runtime objects without creating or returning reusable values.

3. It is destructive or risky as an initial operation, such as delete, deactivate, ban, block, revoke, or remove operations.

4. It depends on complex preconditions that cannot be satisfied at the beginning of fuzzing.

Output requirements:

1. Output valid JSON only.

2. Do not output explanations outside JSON.

3. For each endpoint, include method, path, category, seed_candidate, and reason.
4. Use true or false for seed_candidate.
5. The reason should be short and factual.

Input:
{OPENAPI_ENDPOINT_LIST}

Output format:
[
{
"method": "POST",
"path": "/api/v4/projects",
"category": "ResourceCreate",
"seed_candidate": true,
"reason": "Creates a reusable project resource and may return project identifiers for later requests."
},
{
"method": "GET",
"path": "/api/v4/projects/{id}",
"category": "ResourceRead",
"seed_candidate": false,
"reason": "Requires a dynamic project id that is likely produced by a previous request."
}
]

References

1. Ehsan, A.; Abuhaliqa, M.A.M.; Catal, C.; Mishra, D. Restful api testing methodologies: Rationale, challenges, and solution directions. *Appl. Sci.* **2022**, *12*, 4369. [CrossRef]
2. Ponelat, J.; Rosenstock, L. *Designing APIs with Swagger and OpenAPI. Simon and Schuster*; Manning: Greenwich, CT, USA, 2022.
3. Arcuri, A.; Zhang, M.; Galeotti, J. Advanced white-box heuristics for search-based fuzzing of rest apis. *ACM Trans. Softw. Eng. Methodol.* **2024**, *33*, 1–36. [CrossRef]
4. Golmohammadi, A.; Zhang, M.; Arcuri, A. Testing restful apis: A survey. *ACM Trans. Softw. Eng. Methodol.* **2023**, *33*, 1–41. [CrossRef]
5. Sun, F. Restful api-based software interface testing techniques and common problems analysis. *J. Comb. Math. Comb. Comput.* **2025**, *127*, 7075–7091.
6. APIFuzzer: HTTP API Testing Framework, version 0.9.13. GitHub, Inc.: San Francisco, CA, USA. Available online: https://github.com/KissPeter/APIFuzzer (accessed on 20 May 2026).
7. Qualys Web Application Scanning (WAS). Qualys, Inc.: Foster City, CA, USA. Available online: https://www.qualys.com/apps/web-app-scanning (accessed on 20 May 2026).
8. TnT-Fuzzer: OpenAPI Fuzzer Written in Python. TeeBytes/GitHub, Inc.: San Francisco, CA, USA. Available online: https://github.com/Teebytes/TnT-Fuzzer (accessed on 20 May 2026).
9. Atlidakis, V.; Geambasu, R.; Godefroid, P.; Polishchuk, M.; Ray, B. Pythia: Grammar-based fuzzing of rest apis with coverage-guided feedback and learning-based mutations. *arXiv* **2020**, arXiv:2005.11498.
10. Atlidakis, V.; Godefroid, P.; Polishchuk, M. Restler: Stateful rest api fuzzing. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*; IEEE: Los Alamitos, CA, USA, 2019; pp. 748–758.
11. Atlidakis, V.; Godefroid, P.; Polishchuk, M. Checking security properties of cloud service rest apis. In *2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST)*; Corradini, D., Montolli, Z., Pasqua, M., Ceccato, M., Eds.; IEEE: Los Alamitos, CA, USA, 2020; pp. 387–397.
12. Godefroid, P.; Lehmann, D.; Polishchuk, M. Differential regression testing for rest apis. In *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis*; Association for Computing Machinery: New York, NY, USA, 2020; pp. 312–323.

13. Liu, Y.; Li, Y.; Deng, G.; Liu, Y.; Wan, R.; Wu, R.; Ji, D.; Xu, S.; Bao, M. Morest: Model-based restful api testing with execution feedback. In *Proceedings of the 44th International Conference on Software Engineering*; Association for Computing Machinery: New York, NY, USA; IEEE: Los Alamitos, CA, USA, 2022; pp. 1406–1417.
14. Viglianisi, E.; Dallago, M.; Ceccato, M. Resttestgen: automated black-box testing of restful apis. In Proceedings of the 2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST), Los Alamitos, CA, USA, 24–28 October 2020; pp. 142–152.
15. Yu, X.; Lv, L.; Zhang, Z.; Lan, Y.; Liu, L. Response relation model-driven approach for automated fuzz testing of restful web apis. In *Third International Conference on Informatics, Networking, and Computing (ICINC 2024)*; SPIE: Bellingham, WA, USA, 2025; Volume 13637, p. 1363702.
16. Neumann, A.; Laranjeiro, N.; Bernardino, J. An analysis of public rest web service apis. *IEEE Trans. Serv. Comput.* **2018**, *14*, 957–970. [CrossRef]
17. OpenAPI Specification, version 3.1.0. OpenAPI Initiative, Linux Foundation: San Francisco, CA, USA. Available online: https://swagger.io/specification/ (accessed on 20 May 2026).
18. Arcuri, A. Restful api automated test case generation with evomaster. *ACM Trans. Softw. Eng. Methodol. (TOSEM)* **2019**, *28*, 1–37. [CrossRef]
19. Martin-Lopez, A.; Segura, S.; Ruiz-Cortés, A. Restest: Black-box constraint-based testing of restful web apis. In *International Conference on Service-Oriented Computing*; Springer: Cham, Switzerland, 2020; pp. 459–475.
20. Aschermann, C.; Schumilo, S.; Abbasi, A.; Holz, T. Ijon: Exploring deep state spaces via fuzzing. In *2020 IEEE Symposium on Security and Privacy (SP)*; IEEE: Los Alamitos, CA, USA, 2020; pp. 1597–1612.
21. Lyu, C.; Xu, J.; Ji, S.; Zhang, X.; Wang, Q.; Zhao, B.; Pan, G.; Cao, W.; Chen, P.; Beyah, R. {MINER}: A hybrid {DataDriven} approach for {REST}{API} fuzzing. In *32nd USENIX Security Symposium (USENIX Security 23)*; USENIX Association: Berkeley, CA, USA, 2023; pp. 4517–4534.
22. Deng, G.; Zhang, Z.; Li, Y.; Liu, Y.; Zhang, T.; Liu, Y.; Yu, G.; Wang, D. NAUTILUS: Automated RESTful API vulnerability detection. In *32nd USENIX Security Symposium (USENIX Security 23)*; USENIX Association: Berkeley, CA, USA, 2023; pp. 5593–5609.
23. OWASP API Security Project. The OWASP Foundation Inc.: Wilmington, DE, USA. Available online: https://owasp.org/www-project-api-security/ (accessed on 20 May 2026).
24. Zed Attack Proxy (ZAP), version 2.17.0. ZAP Development Team/Checkmarx: Ramat Gan, Israel. Available online: https://www.zaproxy.org/ (accessed on 20 May 2026).
25. Du, W.; Li, J.; Wang, Y.; Chen, L.; Zhao, R.; Zhu, J.; Han, Z.; Wang, Y.; Xue, Z. Vulnerability-oriented testing for RESTful APIs. In *33rd USENIX Security Symposium (USENIX Security 24)*; USENIX Association: Berkeley, CA, USA, 2024; pp. 739–755.
26. Arcuri, A.; Poth, A.; Rrjolli, O. Introducing black-box fuzz testing for rest apis in industry: Challenges and solutions. In *2025 IEEE Conference on Software Testing, Verification and Validation (ICST)*; IEEE: Los Alamitos, CA, USA, 2025; pp. 382–393.
27. Ou, S.; Li, Y.; Yu, L.; Wei, C.; Wen, T.; Chen, Q.; Chen, Y.; Tang, H.; Pan, Z. Mirrorfuzz: Leveraging llm and shared bugs for deep learning framework apis fuzzing. *IEEE Trans. Softw. Eng.* **2025**, *52*, 360–375. [CrossRef]
28. Yang, L.; Wei, C.; Yang, J.; Xia, W.; Yang, Y.; Luo, Y.; Niyato, D.; Sun, L.; Liu, Z. Fuzzcode: Code large language modelbased fuzz testing for industrial iot programs. *IEEE Internet Things J.* **2025**, *12*, 36842–36851. [CrossRef]
29. Corradini, D.; Montolli, Z.; Pasqua, M.; Ceccato, M. Deeprest: Automated test case generation for rest apis exploiting deep reinforcement learning. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*; IEEE: Los Alamitos, CA, USA, 2024; pp. 1383–1394.
30. Kim, M.; Sinha, S.; Orso, A. Llamaresttest: Effective rest api testing with small language models. *Proc. ACM Softw. Eng.* **2025**, *2*, 465–488. [CrossRef]
31. Stennett, T.; Kim, M.; Sinha, S.; Orso, A. Autoresttest: A tool for automated rest api testing using llms and marl. In *2025 IEEE/ACM 47th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*; IEEE: Los Alamitos, CA, USA, 2025; pp. 21–24.
32. Goby, version Beta 2.2.0; GobySec: Singapore. Available online: https://github.com/gobysec/Goby (accessed on 20 May 2026).
33. Nuclei Templates, version 10.4.3. ProjectDiscovery, Inc.: San Francisco, CA, USA. Available online: https://github.com/projectdiscovery/nuclei-templates (accessed on 20 May 2026).
34. xray, version 1.9.10. Chaitin Tech: Beijing, China. Available online: https://github.com/chaitin/xray (accessed on 20 May 2026).
35. ChatGPT; OpenAI, L.L.C.: San Francisco, CA, USA. Available online: https://chatgpt.com (accessed on 20 May 2026).
36. DeepSeek; Hangzhou DeepSeek Artificial Intelligence Basic Technology Research Co., Ltd.: Hangzhou, Zhejiang, China. Available online: https://www.deepseek.com/ (accessed on 20 May 2026).
37. Han, K.; Wang, Y.; Chen, H.; Chen, X.; Guo, J.; Liu, Z.; Tang, Y.; Xiao, A.; Xu, C.; Xu, Y.; et al. A survey on vision transformer. *IEEE Trans. Pattern Anal. Mach. Intell.* **2022**, *45*, 87–110. [CrossRef] [PubMed]
38. Reyad, M.; Sarhan, A.M.; Arafa, M. A modified adam algorithm for deep neural network optimization. *Neural Comput. Appl.* **2023**, *35*, 17095–17112. [CrossRef]

39. Demissie, B.F.; Tun, Y.N.; Shar, L.K.; Ceccato, M. Vlm-fuzz: Vision language model assisted recursive depth-first search Checked and revised exploration for effective gui testing of android apps. *Empir. Softw. Eng.* **2026**, *31*, 76. [CrossRef]
40. GitLab, version 14.1.0. GitLab Inc.: San Francisco, CA, USA. Available online: https://gitlab.com/gitlab-org/gitlab (accessed on 20 May 2026).
41. WordPress, version 5.8.1. WordPress Foundation: San Francisco, CA, USA. Available online: https://wordpress.org (accessed on 20 May 2026).
42. Bugzilla, version 5.2; The Bugzilla Project: Mountain View, CA, USA. Available online: https://www.bugzilla.org (accessed on 20 May 2026).
43. Corradini, D.; Zampieri, A.; Pasqua, M.; Viglianisi, E.; Dallago, M.; Ceccato, M. Automated black-box testing of nominal and error scenarios in restful apis. *Softw. Test. Verif. Reliab.* **2022**, *32*, e1808. [CrossRef]
44. Zhang, M.; Marculescu, B.; Arcuri, A. Resource and dependency based test case generation for restful web services. *Empir. Softw. Eng.* **2021**, *26*, 76. [CrossRef]
45. Xue, Y.; Ye, J.; Zhang, W.; Sun, J.; Ma, L.; Wang, H.; Zhao, J. xfuzz: Machine learning guided cross-contract fuzzing. *IEEE Trans. Dependable Secur. Comput.* **2022**, *21*, 515–529. [CrossRef]
46. Zhang, A.; Zhang, Y.; Xu, Y.; Wang, C.; Li, S. Machine learning-based fuzz testing techniques: A survey. *IEEE Access* **2023**, *12*, 14437–14454. [CrossRef]
47. Jiang, Y.; Liang, J.; Ma, F.; Chen, Y.; Zhou, C.; Shen, Y.; Wu, Z.; Fu, J.; Wang, M.; Li, S.; et al. When fuzzing meets llms: Challenges and opportunities. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*; Association for Computing Machinery: New York, NY, USA, 2024; pp. 492–496.
48. Zhu, X.; Zhou, W.; Han, Q.-L.; Ma, W.; Wen, S.; Xiang, Y. When software security meets large language models: A survey. *IEEE/CAA J. Autom. Sin.* **2025**, *12*, 317–334. [CrossRef]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.